Bash scripting - Part 3

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

Master Class: Enterprise-Grade Bash Scripting & Systems Engineering

Part 3/3: Production SRE, Diagnostics, Alerting, & Incident Response

---

1. Part Introduction and Scope

In mission-critical, enterprise-grade cloud environments, high availability (HA) depends on real-time visibility, deterministic failure recovery, and zero-dependency diagnostic tools. While higher-level languages like Go or Python dominate modern cloud-native architectures, low-level system diagnostic automation, incident triage, and self-healing mechanisms at the bare-metal, virtual machine, and container runtime layers rely heavily on POSIX-compliant Bash scripting.

When kernel panics, system lockups, OOM events, or network partitions occur, high-level runtime environments (such as Node.js, Python, or JVMs) often become unresponsive or fail due to broken dynamic linkers, missing runtimes, or resource exhaustion. Pure Bash-leveraging built-in shell primitives and core Linux utilities-operates directly against `/proc`, `/sys`, and raw socket APIs.

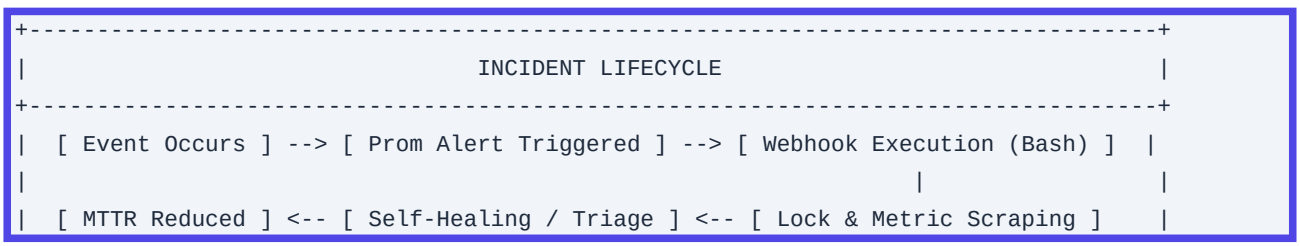
This guide covers:

- Automated Linux diagnostics and real-time triage data collection.
- Native Prometheus textfile exporter integration for low-overhead telemetry.
- Automated incident runbooks, self-healing daemons, and signal-safe execution.
- Advanced production troubleshooting, Root Cause Analysis (RCA) methodology, and enterprise hardening.

---

2. Why SRE-Grade Scripting is Critical for High-Availability Systems

High-Availability (HA) systems (demanding 99.99% or higher uptime) cannot afford manual engineering intervention during initial incident triage. The mean time to detect (MTTD) and mean time to resolve (MTTR) must be minimized through deterministic, scriptable automation.



```
+-------------------------------------------------------------------------------+
```

Key High-Availability Requirements:

1. Zero-Dependency Resilience: During severe system degradation (e.g., dynamic library corruption, package manager locks, or total disk read-only remounts), pre-compiled Go or Python runtimes may fail to load shared objects (`.so`). A statically linked or pure Bash script relying on kernel interfaces (`/proc`, `/sys`) remains execution-ready.

2. Deterministic Resource Overhead: SRE diagnostic scripts must run when CPU or memory is constrained. A heavy Python script initializing a venv can trigger an Out-Of-Memory (OOM) killer event; a tuned Bash script operating via built-ins maintains a minimal footprint.

3. Atomic Telemetry & State Consistency: Diagnostic and metric collection must avoid race conditions. Non-atomic file writes corrupt Prometheus metric scrapes, causing false alarms or monitoring blackouts.

4. Signal-Safe Execution & Cleanup: Interruptions (`SIGTERM`, `SIGINT`) must be trapped to prevent leftover stale lockfiles (`flock`), abandoned background subshells, or zombie processes.

---

3. Real-World Enterprise Use Cases

Use Case 1: Automated K8s Worker Node Self-Healing & Triage Daemon

When a Kubernetes node enters a `NotReady` state due to kernel deadlock, container runtime hang (`containerd`), or file descriptor exhaustion, standard Kubernetes controllers cannot run pod-based jobs on that node. A local daemon written in Bash runs via `systemd` to detect local kernel hung-task alerts, execute a deep network/storage diagnostic, generate a gzipped diagnostic payload to an external S3-compatible object store, drain local workloads safely, and trigger a soft node reboot.

Use Case 2: Incident Emergency Triage Collector (Prometheus Alertmanager Webhook Target)

Upon receiving a high-priority PagerDuty or Alertmanager webhook indicating a database node CPU spike or thread pool exhaustion, an automated agent executes a script that captures process call stacks (`/proc/[pid]/stack`), network socket states via `ss`, dynamic tracing output via `strace`/`bpftrace`, and memory maps (`/proc/[pid]/smaps`). This snapshot is saved before the failing service crashes or auto-scales out, preserving crucial post-mortem RCA artifacts.

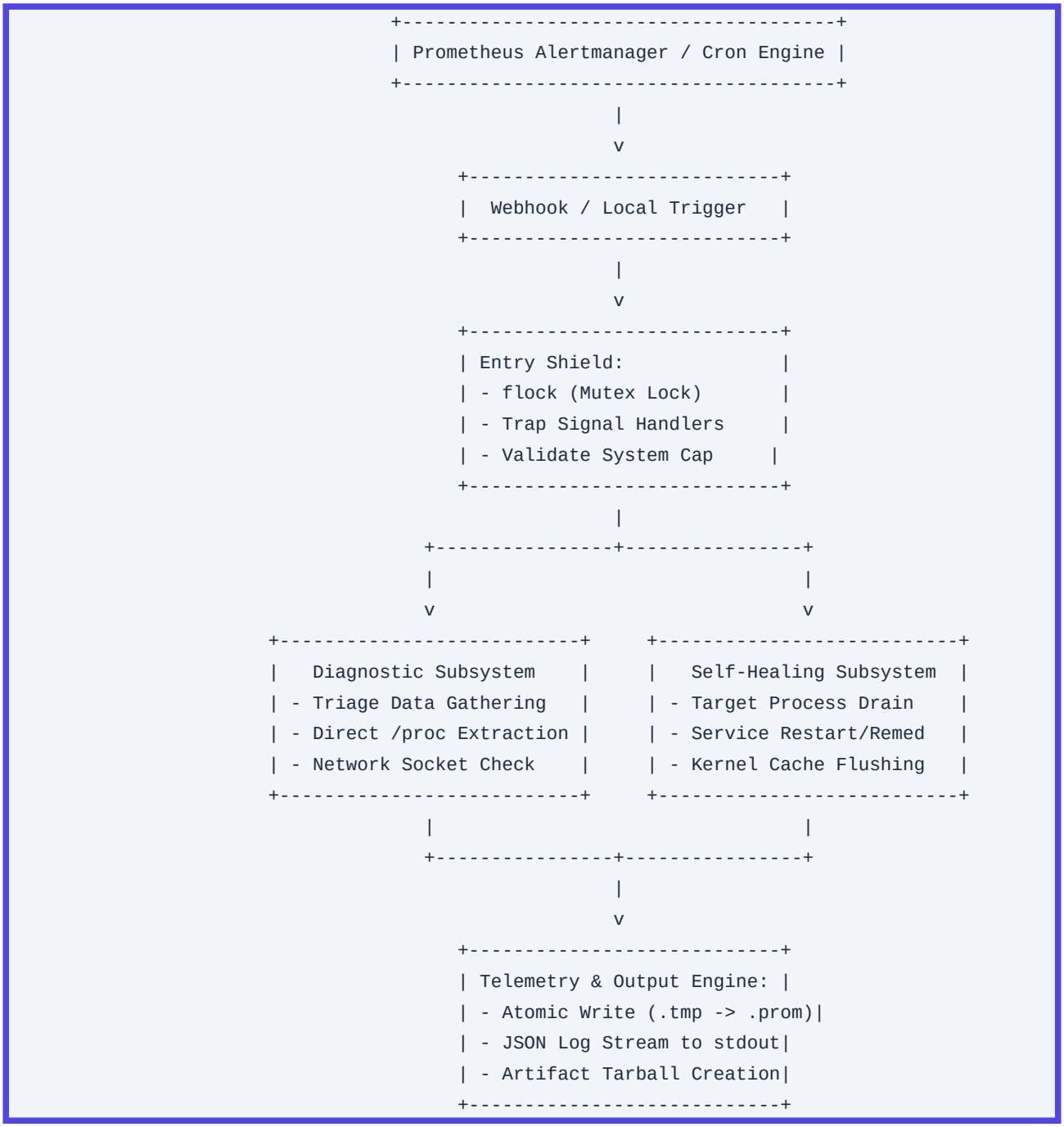
Use Case 3: Zero-Agent Edge Infrastructure Prometheus Metric Generation

In isolated edge computing environments or hardened minimal OS distributions (e.g., Flatcar, Talos, Alpine) where standard monitoring agents cannot be installed, a lightweight Bash script acts as a native metric collector. It extracts system health parameters directly from `/proc`, formats them into valid Prometheus exposition format, and writes them atomically to the node-exporter textfile directory.

---

4. Comprehensive Architecture Explanation

The SRE Automation Engine follows an event-driven architecture designed to isolate execution, guarantee execution exclusivity via non-blocking mutual exclusion (`flock`), enforce strict execution timeouts, and maintain thread-safe metric exposition.



---

5. Components of SRE & Diagnostic Bash Engineering

1. Safety Enforcement Layer: Sets strict interpreter flags (`set -euo pipefail`), defines subshell isolation, establishes custom signal handlers (`trap`), and restricts execution using Linux capabilities (`CAP_SYS_PTRACE`, `CAP_NET_ADMIN`).

2. Locking & Concurrency Controller: Leverages native kernel-level file locks (`flock`) on descriptor files to prevent race conditions or process stackup under heavy node degradation.

3. Kernel Interface Reader (`/proc` & `/sys`): Bypasses slow binary utilities by parsing raw system information directly from kernel exposed pseudo-filesystems (e.g., `/proc/meminfo`, `/proc/net/dev`, `/proc/sys/fs/file-nr`).

4. Diagnostic Extractor: Uses utilities like `ss`, `lsof`, `strace`, and `vmstat` to gather deep system metrics when anomalous states are detected.

5. Atomic Metric Generator: Constructs Prometheus metric structures line-by-line and applies atomic filesystem updates (`mv` / `renameat2`) to directory endpoints scraped by Prometheus `node_exporter`.

6. Structured Logging Unit: Formats all diagnostics into structured JSON strings outputted to `stdout`/`stderr` for log aggregation pipelines (e.g., Vector, Fluentbit, Loki).

---

6. Step-by-Step Production Implementation Guide

The following production script implements an enterprise SRE Diagnostic & Self-Healing Daemon. It collects critical system diagnostics, handles process exhaustion safely, generates Prometheus textfile metrics atomically, and triggers automated remediation if thresholds are exceeded.

Save this script as `/usr/local/bin/sre_node_triage.sh`.

```
#!/usr/bin/env bash
# ================================================================================
# Script Name : sre_node_triage.sh
# Enterprise SRE Automated Diagnostic & Self-Healing Engine
# Target OS   : Linux (RHEL/SLES/Debian/Ubuntu)
# ================================================================================

# Strict Execution Mode
set -euo pipefail
IFS=$'\n\t'

# Global Configuration Parameters
readonly SCRIPT_NAME="$(basename "${BASH_SOURCE[0]}")"
readonly LOCK_FILE="/var/run/${SCRIPT_NAME}.lock"
readonly PROM_TEXTFILE_DIR="/var/lib/prometheus/node-exporter"
readonly PROM_METRIC_FILE="${PROM_TEXTFILE_DIR}/sre_node_diagnostics.prom"
readonly DIAGNOSTIC_OUTPUT_DIR="/var/log/sre_triages"
readonly LOG_MAX_AGE_DAYS=7
```

```
readonly MEM_THRESHOLD_PCT=90
readonly FD_THRESHOLD_PCT=80

# Signal Trap & Safe Exit Initialization
cleanup() {
    local exit_code=$?
    trap - EXIT SIGINT SIGTERM

    # Remove lock file descriptor lock handle safely
    if [[ -d "${DIAGNOSTIC_TMP_DIR:-}" ]]; then
        rm -rf "${DIAGNOSTIC_TMP_DIR}"
    fi

    log_json "INFO" "Execution completed with exit code: ${exit_code}"
    exit "${exit_code}"
}

trap cleanup EXIT SIGINT SIGTERM

# Structured JSON Logging Function
log_json() {
    local severity="$1"
    local message="$2"
    local timestamp
    timestamp="$(date -u +"%Y-%m-%dT%H:%M:%SZ")"

    printf '{"timestamp":"%s","severity":"%s","script":"%s","pid":%d,"message":"%s"}\n' \
        "${timestamp}" "${severity}" "${SCRIPT_NAME}" "$$" "${message//\"/\\\"}"
}

# Enforce Mutual Exclusion using flock
exec 200>"${LOCK_FILE}"
if ! flock -n 200; then
    log_json "WARN" "Another instance of ${SCRIPT_NAME} is running. Exiting immediately."
    exit 0
fi

# Ensure Target Paths Exist
mkdir -p "${PROM_TEXTFILE_DIR}" "${DIAGNOSTIC_OUTPUT_DIR}"
DIAGNOSTIC_TMP_DIR="$(mktemp -d /tmp/sre_triage.XXXXXX)"

# System Capability Verification
check_prerequisites() {
    local required_cmds=("ss" "awk" "lsof" "gzip" "vmstat")
    for cmd in "${required_cmds[@]}"; do
        if ! command -v "${cmd}" &>/dev/null; then
            log_json "ERROR" "Prerequisite command missing from PATH: ${cmd}"
            exit 1
```

```
        fi
    done
}

# Kernel Diagnostic Data Extraction (/proc Direct Access)
collect_memory_pressure() {
    local mem_total mem_available mem_used_pct
    mem_total=$(awk '/MemTotal:/ {print $2}' /proc/meminfo)
    mem_available=$(awk '/MemAvailable:/ {print $2}' /proc/meminfo)

    mem_used_pct=$(( (mem_total - mem_available) * 100 / mem_total ))
    echo "${mem_used_pct}"
}

collect_fd_usage() {
    local fd_allocated fd_max fd_used_pct
    read -r fd_allocated _ fd_max < /proc/sys/fs/file-nr
    fd_used_pct=$(( fd_allocated * 100 / fd_max ))
    echo "${fd_used_pct}"
}

# Generate Prometheus Metrics Atomically
write_prometheus_metrics() {
    local mem_pct="$1"
    local fd_pct="$2"
    local execution_status="$3"
    local temp_prom_file="${PROM_METRIC_FILE}.tmp.${$}"

    cat <<EOF > "${temp_prom_file}"
# HELP sre_node_memory_utilization_percent Current memory usage calculated via /proc/meminfo.
# TYPE sre_node_memory_utilization_percent gauge
sre_node_memory_utilization_percent ${mem_pct}

# HELP sre_node_file_descriptors_utilization_percent Allocated file descriptors vs total
allowed limit.
# TYPE sre_node_file_descriptors_utilization_percent gauge
sre_node_file_descriptors_utilization_percent ${fd_pct}

# HELP sre_node_diagnostic_execution_status Status of the SRE diagnostic execution
(1=Success, 0=Failure).
# TYPE sre_node_diagnostic_execution_status gauge
sre_node_diagnostic_execution_status ${execution_status}

# HELP sre_node_diagnostic_last_run_timestamp Unix timestamp of last execution.
# TYPE sre_node_diagnostic_last_run_timestamp gauge
sre_node_diagnostic_last_run_timestamp $(date +%s)
EOF
```

```
    # Atomic Rename Operation
    mv -f "${temp_prom_file}" "${PROM_METRIC_FILE}"
    chmod 0644 "${PROM_METRIC_FILE}"
}

# Deep Diagnostic Artifact Collector
capture_incident_snapshot() {
    local reason="$1"
    local timestamp
    timestamp="$(date +"%Y%m%d_%H%M%S")"
    local archive_name="${DIAGNOSTIC_OUTPUT_DIR}/triage_snapshot_${timestamp}.tar.gz"

    log_json "WARN" "High pressure detected (${reason}). Capturing deep triage artifacts."

    # Collect stack traces, socket tables, top resource consumers
    {
        echo "=== TOP PROCESSES BY CPU ==="
        ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -n 20

        echo -e "\n=== TOP PROCESSES BY MEMORY ==="
        ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%mem | head -n 20

        echo -e "\n=== NETWORK SOCKET SUMMARY ==="
        ss -s

        echo -e "\n=== OPEN FILE DESCRIPTORS (TOP CONSUMERS) ==="
        lsof +c 15 | awk '{print $1}' | sort | uniq -c | sort -nr | head -n 15

        echo -e "\n=== KERNEL DMESG RECENT TAIL ==="
        dmesg --human --level=err,warn | tail -n 50

        echo -e "\n=== VMSTAT DYNAMICS ==="
        vmstat 1 5
    } > "${DIAGNOSTIC_TMP_DIR}/system_snapshot.txt"

    # Compress artifacts safely
    tar -czf "${archive_name}" -C "${DIAGNOSTIC_TMP_DIR}" system_snapshot.txt
    log_json "INFO" "Diagnostic snapshot successfully archived to ${archive_name}"
}

# Main Execution Flow
main() {
    log_json "INFO" "Starting node diagnostic sweep."
    check_prerequisites

    local mem_pct fd_pct execution_success=1
    mem_pct=$(collect_memory_pressure)
    fd_pct=$(collect_fd_usage)
```

```
    log_json "INFO" "Resource Check: Memory Usage=${mem_pct}%, File Descriptors
Usage=${fd_pct}%"

    # Evaluate Threshold Exceedance
    if (( mem_pct >= MEM_THRESHOLD_PCT )); then
        capture_incident_snapshot "Memory_Threshold_Exceeded_${mem_pct}_Percent"
        # Enterprise Remediation Hook (e.g., flush OS pagecache safely if critical)
        sync && echo 3 > /proc/sys/vm/drop_caches || log_json "ERROR" "Failed to clear page
cache"
    fi

    if (( fd_pct >= FD_THRESHOLD_PCT )); then
        capture_incident_snapshot "FD_Threshold_Exceeded_${fd_pct}_Percent"
    fi

    # Rotate old artifacts to prevent local disk space exhaustion
    find "${DIAGNOSTIC_OUTPUT_DIR}" -type f -name "triage_snapshot_*.tar.gz" -mtime
+"${LOG_MAX_AGE_DAYS}" -delete

    write_prometheus_metrics "${mem_pct}" "${fd_pct}" "${execution_success}"
    log_json "INFO" "Diagnostic sweep complete."
}

main "$@"
```

---

7. Deep Dive: Linux Diagnostic CLI Commands & Flags

SRE scripts invoke Linux diagnostics to inspect system state. Understanding these specific flags is critical for accurate troubleshooting without introducing unnecessary overhead.

1. `ss -tlpnmsi` (Socket Statistics)

- `-t`: Displays TCP sockets only.
- `-l`: Shows listening sockets (omits established connections unless configured otherwise).
- `-p`: Identifies the process ID (PID) and executable name using the socket (requires root/CAP_NET_ADMIN).
- `-n`: Disables hostname resolution to prevent DNS lookups from blocking script execution during network incidents.
- `-m`: Displays socket memory usage (skmem, rmem, wmem allocations), essential for diagnosing socket buffer exhaustions.
- `-s`: Outputs high-level socket stat summaries directly from kernel structures.

- `-i`: Exposes internal TCP metrics (RTT, congestion window `cwnd`, path MTU, retransmits).

2. `lsof +L -p <PID>` (Open File Descriptor Analysis)

- `+L`: Disables the lookup of directory link counts, highlighting files that have been deleted from the filesystem but remain held open in memory by a running process (a common cause of unexplained disk space usage).
- `-p <PID>`: Filters file descriptors exclusively for the specified process group ID, avoiding long-running sweeps across all system PIDs.

3. `strace -ff -T -tt -s 1024 -p <PID>` (System Call Tracing)

- `-ff`: Follows child forks and dumps system calls for each thread into distinct output files (`filename.PID`).
- `-T`: Logs the time spent inside each system call, isolating blocking I/O calls (`read`, `write`, `epoll_wait`).
- `-tt`: Microsecond-precision wall-clock timestamping for every syscall line.
- `-s 1024`: Increases string printing limit from default 32 bytes to 1024 bytes (useful for inspecting SQL queries or HTTP headers passed to `read`/`write`).
- `-p <PID>`: Attaches directly to a running target process ID.

4. `vmstat -SM 1 5` (Virtual Memory Statistics)

- `-S M`: Displays output units in Megabytes rather than default raw blocks.
- `1`: Interval in seconds between sampling iterations.
- `5`: Count of reports generated before execution terminates.
- *Key Fields*: `r` (runnable queued processes), `b` (uninterruptible sleep processes blocked on I/O), `si`/`so` (swap-in/swap-out activity indicating physical memory pressure).

5. `iostat -xz 1 5` (Extended I/O Metrics)

- `-x`: Outputs extended statistics including device queue lengths, await times, and service times.
- `-z`: Omits output for idle storage devices, reducing log noise in systems with hundreds of attached block volumes.
- *Key Fields*: `%util` (bandwidth utilization of device), `await` (average time in ms for I/O requests to be served), `w_await`/`r_await` (isolates write latency from read latency).

---

8. Production Configuration Examples

Systemd Service Unit File

Create `/etc/systemd/system/sre-node-triage.service`:

```
[Unit]
Description=SRE Enterprise Node Triage and Self-Healing Engine
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=/usr/local/bin/sre_node_triage.sh
User=root
Group=root

# Security Hardening Directives
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
ProtectKernelTunables=false
ProtectKernelModules=true
ProtectControlGroups=true
MemoryDenyWriteExecute=true
CapabilityBoundingSet=CAP_SYS_PTRACE CAP_NET_ADMIN CAP_DAC_READ_SEARCH CAP_FOWNER
ReadWritePaths=/var/lib/prometheus/node-exporter /var/log/sre_triages /var/run /tmp

[Install]
WantedBy=multi-user.target
```

Systemd Timer Unit File

Create `/etc/systemd/system/sre-node-triage.timer`:

```
[Unit]
Description=Run SRE Node Triage Script Every 2 Minutes

[Timer]
OnBootSec=1min
OnUnitActiveSec=2min
AccuracySec=1us

[Install]
WantedBy=timers.target
```

Enterprise Prometheus Alerting Rules (`/etc/prometheus/rules/sre_alerts.yaml`)

```
groups:
  - name: SRE_Automation_Monitoring
    rules:
      - alert: SRENodeHighMemoryUsage
        expr: sre_node_memory_utilization_percent > 90
```

```
        for: 2m
        labels:
          severity: critical
          tier: infrastructure
        annotations:
          summary: "Node memory pressure critical on {{ $labels.instance }}"
          description: "Memory utilization reached {{ $value }}%. Automated remediation
initiated."

      - alert: SREDiagnosticDaemonFailing
        expr: sre_node_diagnostic_execution_status == 0
        for: 5m
        labels:
          severity: warning
          tier: observability
        annotations:
          summary: "SRE diagnostic script failure on {{ $labels.instance }}"
          description: "The automated Bash diagnostic script failed to execute cleanly."

      - alert: SREDiagnosticStaleMetrics
        expr: (time() - sre_node_diagnostic_last_run_timestamp) > 300
        for: 1m
        labels:
          severity: warning
          tier: observability
        annotations:
          summary: "SRE diagnostic metrics are stale on {{ $labels.instance }}"
          description: "The diagnostic script has not updated metrics in > 5 minutes."
```

---

9. Security Considerations & Hardening Best Practices

1. Capability Restriction (Avoiding Unrestricted Root Executions)

Do not execute SRE scripts as full root if only specific administrative operations are required. Use systemd capability bounding or `sudoers` fine-grained restrictions:

```
# Sudoers configuration for specific binary privilege escalation
sre_agent ALL=(ALL) NOPASSWD: /usr/bin/dmesg, /usr/sbin/ss, /usr/bin/gcore
```

2. Command Injection Prevention via Strict Input Sanitization

Never pass raw external inputs (such as incoming HTTP Webhook JSON payloads) directly into dynamic evaluation statements (`eval`) or unquoted variable expressions.

```
# VULNERABLE:
# eval "process_id=$INCOMING_JSON_PID"

# HARDENED:
if [[ "${INCOMING_JSON_PID}" =~ ^[0-9]+$ ]]; then
    target_pid="${INCOMING_JSON_PID}"
else
    log_json "ERROR" "Invalid PID provided: ${INCOMING_JSON_PID}"
    exit 1
fi
```

3. Secure Temporary File and Directory Creation

Never write diagnostic traces to predictable `/tmp` paths, which leaves systems vulnerable to symlink attacks (`TOCTOU`). Always use `mktemp`:

```
# HARDENED Directory Allocation
SECURE_TMP="$(mktemp -d -p /tmp sre_diag_XXXXXX)"
# Set restrictive permissions immediately
chmod 0700 "${SECURE_TMP}"
```

4. PATH Sanitization

Always set an explicit, immutable `PATH` variable at the top of production SRE scripts to prevent hijacking via path prepending in multi-user environments:

```
readonly PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
export PATH
```

---

10. Observability & Monitoring Considerations

Native Prometheus Textfile Metric Protocol Rules

When creating custom `.prom` metrics with Bash, follow the official Prometheus exposition format:

1. Every metric must include `# HELP` and `# TYPE` declarations.

2. Metric values must end with a trailing newline.

3. Metric updates must be atomic. Write metric values to a `.tmp` file on the same filesystem mount and rename it via `mv`. Writes directly to the target file risk race conditions where Prometheus scrapes truncated data.

Structured Log Aggregation Strategy

Transform standard Bash output lines into JSON objects. Modern log collectors (Vector, FluentBit, Logstash) process structured key-value maps significantly faster than unstructured string regex parsing.

```
# Helper function to write structured logs to stdout
emit_log() {
    local level="$1"
    local evt="$2"
    local details="$3"

    jq -n \
      --arg time "$(date -u +"%Y-%m-%dT%H:%M:%SZ")" \
      --arg level "$level" \
      --arg event "$evt" \
      --arg details "$details" \
      --arg node "${NODE_NAME:-$(hostname)}" \
      '{timestamp: $time, level: $level, event: $event, details: $details, node: $node}'
}
```

---

11. Common Troubleshooting Scenarios & RCA Protocols

Scenario 1: Zombie Process Explosion during Concurrent Scans

- **Symptom**: Node process count spikes to `kernel.pid_max` limit, preventing new processes from spawning. `ps` shows hundreds of `<defunct>` entries created by the Bash script.
- **Root Cause**: Diagnostic scripts executing subshells in background loops (e.g., `cmd &`) without waiting for child execution contexts to terminate, or failing to capture `SIGCHLD`.
- **RCA & Fix**:

Always enforce subshell limits and invoke the `wait` builtin command:

```
# Incorrect Background Loop
for ip in "${IP_LIST[@]}"; do
    ping -c 1 "$ip" & # Generates untracked subshells
done

# Correct Controlled Execution Block
for ip in "${IP_LIST[@]}"; do
    ping -c 1 "$ip" &
    # Allow a maximum of 10 concurrent processes
    if [[ $(jobs -r -p | wc -l) -ge 10 ]]; then
        wait -n # Wait for any single child process to exit
    fi
done
wait # Wait for remaining background jobs to exit
```

Scenario 2: Unlinked File Descriptor Disk Space Leak

- **Symptom**: `df -h` reports `/var/log` is 100% full, but running `du -sh /var/log/*` shows minimal file space consumed.
- **Root Cause**: A diagnostic runbook redirected process output to a log file, which was subsequently deleted by a log rotation cron job while the process still held an active open file descriptor.
- **RCA & Fix**:

Run the following diagnostic script to locate deleted open files and restart the responsible service process:

```
# Locate PIDs holding handles to unlinked files > 100MB
lsof +L1 | awk '$7 > 104857600 {print $1, $2, $4, $7, $10}'
```

---

12. Common Production Scripting Pitfalls

| Pitfall | Technical Root Cause | Production Impact | Remediation Strategy |

| :--- | :--- | :--- | :--- |

| Unquoted Array Expansion | `${ARRAY[@]}` vs `"${ARRAY[@]}"` | Word splitting breaks path parameters containing spaces or shell metacharacters | Always double-quote array expansions: `"${ARRAY[@]}"` |

| Reliance on Pipe Without `pipefail` | Shell defaults to exit code of *last* command in a pipe sequence | Failure in upstream diagnostic command (`grep` or `awk`) goes undetected | Place `set -o pipefail` at the script header |

| In-Place Metric File Edits | Using `sed -i` or `echo >` directly on `.prom` exporter path | Prom-exporter reads empty/partially written file, drops metrics | Write to temporary file, then use atomic `mv` |

| Memory Leak via Variable Accumulation | Appending logs to internal Bash string variable inside infinite loop | Script slowly consumes host memory until killed by OS OOM | Stream logs directly to stdout/file; avoid accumulating state in long-lived scripts |

---

13. Enterprise Recommendations & Performance Tuning

Process Subshell Overhead Reduction

Spawning external binaries (`sed`, `grep`, `cut`, `awk`) inside tight loops creates expensive `fork()` and `execve()` system calls that degrade CPU performance. Replace external binary calls with native Bash parameter expansion wherever possible.

```
# SLOW (Spawns 2 subshells per iteration):
for path in /var/log/app/*.log; do
    filename=$(echo "$path" | cut -d'/' -f5)
    ext=$(echo "$path" | sed 's/.*\.//')
```

```
done

# FAST (Uses native Bash built-in execution):
for path in /var/log/app/*.log; do
    filename="${path##*/}"
    ext="${path##*.}"
done
```

`/proc` Kernel Reader Optimization Benchmark

To achieve low-overhead metrics collection, read directly from `/proc` rather than relying on higher-level tools:

```
# HIGH OVERHEAD: Invokes top, parses text output
cpu_idle=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%* id.*/\1/")

# OPTIMIZED: Reads raw CPU tick counters directly from kernel memory interface
read -r cpu user nice system idle iowait irq softirq _ < /proc/stat
```

---

14. Advanced Concepts

1. Anonymous Named Pipes (FIFOs) for Non-Blocking Process IPC

For complex diagnostics requiring parallel data processing without writing temporary files to disk, create an in-memory named pipe:

```
# Create transient FIFO in virtual filesystem
RAM_FIFO="$(mktemp -u -p /tmp fifo.XXXXXX)"
mkfifo "${RAM_FIFO}"

# Attach file descriptors 3 and 4 to bidirectional FIFO
exec 3<>"${RAM_FIFO}"
rm -f "${RAM_FIFO}" # Unlink handle immediately; memory buffer persists via FD 3

# Producer writes data into descriptor 3
printf "CPU_LOAD=0.85\nMEM_FREE=1024\n" >&3

# Consumer reads data from descriptor 3
read -r metric_line <&3
echo "Ingested: ${metric_line}"

# Close custom descriptors
exec 3>&-
```

2. Trap Management for Dynamic Subshell Graphs

When a master script spawns background diagnostics across multiple child process groups, catching `SIGTERM` requires forwarding the signal down the process tree to avoid orphan background processes:

```
#!/usr/bin/env bash
set -euo pipefail

# Trap signal and forward it to child process group
trap 'kill -TERM -$$; exit 1' SIGINT SIGTERM

# Execute parallel background tasks
sleep 300 &
sleep 300 &

# Wait for all processes in background job control pool
wait
```

---

15. Integrations with DevOps Tools

1. Kubernetes Container Lifecycle Hooks (`preStop`)

Use hardened inline scripts within `preStop` pod lifecycles to guarantee clean application shutdowns and flush connection pools before `SIGKILL` is sent.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: transactional-api
spec:
  template:
    spec:
      containers:
        - name: api
          image: internal-registry.enterprise.io/api:v2.4
          lifecycle:
            preStop:
              exec:
                command:
                  - /bin/bash
                  - -c
                  - |
                    set -euo pipefail
                    # Drain Active Connections Signal
                    touch /tmp/drain_started
```

```
                    # Query process socket states until active pool drops to zero
                    while [ $(ss -H -t state established '( sport = :8080 )' | wc -l) -gt 0
]; do
                        sleep 1
                    done
                    rm -f /tmp/drain_started
```

2. Terraform Custom `local-exec` Provisioner Safeguards

Ensure `local-exec` provisioner scripts fail fast and output explicit diagnostic logs to Terraform execution streams:

```
resource "null_resource" "cluster_post_configuration" {
  provisioner "local-exec" {
    interpreter = ["/bin/bash", "-e", "-o", "pipefail", "-c"]
    command     = <<EOT
      /usr/local/bin/validate_cluster_mesh.sh \
        --endpoint "${aws_lb.internal.dns_name}" \
        --timeout 300
    EOT
  }
}
```

---

16. Tooling Matrix: Tooling Comparison

| Diagnostic Approach | Native Bash Engine | Python SRE Scripts | Go Diagnostic Binaries |

| :--- | :--- | :--- | :--- |

| Startup Overhead | Sub-millisecond (< 2ms) | ~30ms - 150ms (Interpreter initialization) | Sub-millisecond (< 1ms) |

| External Dependencies | Standard Linux Coreutils | Python Runtime, `pip` packages, venvs | Statically linked single binary |

| Execution Resilience | High (Works during dynamic library failure) | Medium (Fails if glibc/python core damaged) | Very High (Self-contained statically built) |

| Parsing Complex Formats | Complex (Requires `jq`/`awk`) | Native (Built-in `json`, `yaml` libraries) | Native (Strongly typed parsing) |

| Memory Footprint | Extremely low (~1-3MB) | Moderate (~15-50MB) | Low (~5-15MB) |

| Best Used For | Zero-dependency emergency triage, `/proc` extraction | Parsing complex REST API payloads, multi-cloud SDK calls | High-concurrency background metrics daemons |

---

17. SRE Diagnostics & Scripting Cheat Sheet

Critical System Kernel Paths (`/proc`)

- `/proc/sys/fs/file-nr`: File descriptor usage statistics (Allocated, Unused, Max Allowed).
- `/proc/stat`: CPU tick distribution across modes (user, system, idle, iowait).
- `/proc/meminfo`: Real-time OS memory breakdown (MemAvailable, SwapFree, Buffers, Cached).
- `/proc/net/dev`: Network adapter byte/packet counters and drop rates.
- `/proc/[pid]/cmdline`: Arguments passed to the target process.
- `/proc/[pid]/fd/`: Directory of active open file descriptors for the given PID.

Atomic Operations & Execution Snippets

```
# Non-blocking Exclusion Lock
exec 9>>/var/run/script.lock
flock -n 9 || { echo "Locked"; exit 0; }

# Safe Atomic File Write
printf "metric_value %s\n" "$val" > file.prom.tmp && mv -f file.prom.tmp file.prom

# Signal Handler Trap Implementation
trap 'exit_code=$?; cleanup_handler; exit $exit_code' EXIT SIGINT SIGTERM

# Direct Network Port Availability Test via Native Bash Socket
timeout 2 bash -c '</dev/tcp/127.0.0.1/5432' && echo "Port Open" || echo "Port Closed"
```

---

18. Final Learning Summary

Mastering SRE-grade Bash scripting requires shifting from simple administrative task automation to building resilient, low-overhead diagnostic engines. System engineers must prioritize resilience, safety, and strict execution guarantees:

1. System Safety First: Enforce `set -euo pipefail`, handle process interrupts with explicit `trap` signals, use atomic operations for file modifications, and lock concurrent script invocations using kernel-level `flock`.

2. Interact Directly with Kernel Interfaces: Minimize reliance on external binary utilities within high-frequency monitoring loops. Read system metrics directly from `/proc` and `/sys` to maintain low resource overhead during outage events.

3. Observability Integration: Ensure all scripts output logs in structured format (JSON) and publish health parameters using atomic file operations formatted for Prometheus scrape endpoints.

4. Defense-in-Depth Security: Follow the principle of least privilege. Limit execution capabilities, restrict script

execution directories, set explicit binary paths (`PATH`), and carefully validate external inputs to prevent command injection.

Q41. You're observing high CPU utilization on a critical application server, but the application itself reports healthy. How would you use Bash scripting to perform deep-dive diagnostics and identify the root cause, considering the application is Java-based and running within a Docker container?

Detailed Answer:

When an application reports healthy but system-level metrics (like CPU) are high, it often indicates resource contention, inefficient background processes, or even issues within the underlying runtime (JVM in this case) or operating system interactions. Bash scripting becomes invaluable here for rapidly collecting granular diagnostic data from various layers.

First, identify the container and the Java process within it. We can use `docker ps` to get the container ID/name, then `docker exec` to run commands inside.

To pinpoint the high CPU threads within the JVM, we'd typically use a combination of `top` (or `htop`), `ps`, `jstack`, and `perf`. However, `jstack` requires the JDK to be present in the container, which isn't always the case in production-slimmed images. A more robust approach involves correlating OS-level thread IDs with JVM-internal thread IDs.

The diagnostic script would perform these steps:

1. Identify the Java Process: `docker exec <container_id> ps aux | grep java | grep -v grep | awk '{print $2}'` to get the PID of the JVM inside the container.

2. Monitor Top CPU Threads: Use `docker exec <container_id> top -H -b -n 1 | head -n 20` to get the top processes/threads consuming CPU. Look for thread IDs (LWP column in `top`).

3. Convert Thread IDs: The LWP from `top` is a decimal value. JVM stack dumps report thread IDs in hexadecimal. Convert the top N CPU-consuming LWPs to hex: `printf "0x%x\n" <decimal_LWP>`.

4. Capture JVM Thread Dump: Execute `docker exec <container_id> kill -3 <java_pid>` (SIGQUIT) which forces the JVM to print a full thread dump to its standard error or log file, without terminating the process. This is the `jstack` equivalent when `jstack` isn't available.

5. Retrieve Logs: Immediately after the SIGQUIT, fetch the container logs: `docker logs <container_id> --since "5s"`.

6. Correlate: Grep the logs for the hexadecimal thread IDs identified in step 3. The stack trace associated with these IDs will reveal what the high-CPU threads are executing. This could be garbage collection, specific application logic, or native library calls.

7. System-level Context: Beyond the container, check the host's resource utilization: `top`, `vmstat`, `iostat` to rule out host-level contention or I/O bottlenecks impacting the container. For network issues, `ss -s` or `netstat -s` for socket statistics.

Production Scenario / Practical Example:

Let's say we have a container named `my-java-app-01`.

```
#!/bin/bash
CONTAINER_NAME="my-java-app-01"
LOG_FILE="/tmp/${CONTAINER_NAME}_diagnostics_$(date +%Y%m%d%H%M%S).log"
MAX_THREADS_TO_INVESTIGATE=5

echo "--- Starting diagnostics for container: $CONTAINER_NAME at $(date) ---" | tee -a
"$LOG_FILE"

# 1. Get Java PID inside the container
JAVA_PID=$(docker exec "$CONTAINER_NAME" ps aux | grep java | grep -v grep | awk '{print $2}'
| head -n 1)
if [ -z "$JAVA_PID" ]; then
    echo "ERROR: Java process not found in container $CONTAINER_NAME." | tee -a "$LOG_FILE"
    exit 1
fi
echo "Java PID inside container: $JAVA_PID" | tee -a "$LOG_FILE"

echo "--- Top CPU threads (inside container) ---" | tee -a "$LOG_FILE"
TOP_OUTPUT=$(docker exec "$CONTAINER_NAME" top -H -b -n 1 | head -n 20)
echo "$TOP_OUTPUT" | tee -a "$LOG_FILE"

# Extract LWPs (Light-Weight Processes/threads) from top output
# Assuming the LWP column is the 1st after stripping headers
TOP_LWPS=$(echo "$TOP_OUTPUT" | awk 'NR>7 {print $1}' | head -n
"$MAX_THREADS_TO_INVESTIGATE")
if [ -z "$TOP_LWPS" ]; then
    echo "WARNING: Could not extract LWPs from top output." | tee -a "$LOG_FILE"
fi

echo "--- Generating JVM thread dump (SIGQUIT) ---" | tee -a "$LOG_FILE"
# Send SIGQUIT to generate a thread dump. Output goes to stderr/logs.
docker exec "$CONTAINER_NAME" kill -3 "$JAVA_PID"
sleep 5 # Give JVM time to write the dump

echo "--- Fetching container logs for thread dump ---" | tee -a "$LOG_FILE"
# Fetch recent logs which should contain the thread dump
THREAD_DUMP_LOGS=$(docker logs "$CONTAINER_NAME" --since "10s")
echo "$THREAD_DUMP_LOGS" | tee -a "$LOG_FILE"

echo "--- Correlating high-CPU threads with stack traces ---" | tee -a "$LOG_FILE"
for LWP in $TOP_LWPS; do
    HEX_LWP=$(printf "0x%x\n" "$LWP")
    echo "Investigating LWP (dec): $LWP, (hex): $HEX_LWP" | tee -a "$LOG_FILE"
    # Find stack traces containing the hex LWP (thread ID)
    echo "$THREAD_DUMP_LOGS" | awk "/nid=$HEX_LWP/,/^$/" | tee -a "$LOG_FILE"
done
```

```
echo "--- Host-level resource usage (example) ---" | tee -a "$LOG_FILE"
# Assuming this script runs on the Docker host
echo "Host top -b -n 1:" | tee -a "$LOG_FILE"
top -b -n 1 | head -n 20 | tee -a "$LOG_FILE"
echo "Host vmstat 1 5:" | tee -a "$LOG_FILE"
vmstat 1 5 | tee -a "$LOG_FILE"

echo "--- Diagnostics complete. Results in $LOG_FILE ---" | tee -a "$LOG_FILE"
```

This script automates the collection and initial correlation, providing a solid foundation for diagnosing the application's high CPU usage. You'd typically find patterns like "waiting on monitor" (contention), "doing garbage collection," or specific application methods consuming CPU.

Q42. Describe how you would use Bash scripting to implement a custom log analysis pipeline that detects specific error patterns in application logs and automatically triggers a Prometheus alert if a threshold is exceeded within a defined time window.

Detailed Answer:

Implementing a custom log analysis pipeline with Bash for Prometheus alerting involves several steps: monitoring log files, parsing relevant data, counting occurrences, and exposing these counts as metrics consumable by Prometheus. This is often achieved using the Prometheus Node Exporter's Textfile Collector.

The Bash script would perform the following:

1. Log File Tail/Ingestion: Continuously or periodically read new log entries. For real-time analysis, `tail -F` is useful, though for periodic checks, parsing log files since the last check is more robust (e.g., using `find` with `-mtime` or `logrotate` archives). For containerized apps, `kubectl logs` or `docker logs` would be used.

2. Pattern Matching: Use `grep` or `awk` to filter log lines matching specific error patterns (e.g., "ERROR", "java.lang.NullPointerException", "failed to connect"). Regular expressions are key here for flexibility.

3. Aggregation/Counting: Count the occurrences of these patterns within a specified time window. This could involve `awk` for complex parsing and counting, or simply `grep -c`.

4. Metric Formatting: Format the aggregated counts into Prometheus exposition format. This means `metric_name{label="value"} value`. The Node Exporter Textfile Collector expects files in a specific directory (e.g., `/var/lib/node_exporter/textfile_collector`) with this format.

5. Atomic Update: Write the formatted metrics to a temporary file, then atomically move it to the Node Exporter's collection directory. This prevents Prometheus from reading incomplete or corrupted metric files.

6. Scheduling: Schedule the script to run periodically (e.g., every 30-60 seconds) using `cron` or a `systemd.timer`.

Prometheus Alerting:

Once the metrics are exposed (e.g., `application_error_count_total{app="my-service"} 15`), a Prometheus alerting rule would be configured in `alert.rules.yml` or a similar file:

```
groups:
- name: application-errors
  rules:
  - alert: HighApplicationErrors
    expr: sum(rate(application_error_count_total{app="my-service"}[5m])) by (app) > 5
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "High number of application errors in {{ $labels.app }}"
      description: "{{ $labels.app }} is experiencing more than 5 errors/minute for 2
minutes. Investigate logs for patterns."
```

This rule would trigger an alert if the average rate of errors over the last 5 minutes exceeds 5 errors per minute, sustained for 2 minutes.

Production Scenario / Practical Example:

Consider a microservice `auth-service` whose logs are written to `/var/log/auth-service/current.log`. We want to alert if "AuthenticationFailed" errors exceed 10 in a 1-minute window.

```
#!/bin/bash
# Script: /usr/local/bin/collect_auth_errors.sh

LOG_FILE="/var/log/auth-service/current.log"
METRICS_DIR="/var/lib/node_exporter/textfile_collector"
METRIC_NAME="auth_service_auth_failed_total"
APP_NAME="auth-service"
TEMP_METRIC_FILE="${METRICS_DIR}/auth_errors.prom.tmp"
FINAL_METRIC_FILE="${METRICS_DIR}/auth_errors.prom"

# Ensure the metrics directory exists
mkdir -p "$METRICS_DIR"

# Get the timestamp for 1 minute ago in epoch seconds
ONE_MINUTE_AGO=$(date -d "1 minute ago" +%s)

# Count "AuthenticationFailed" errors in the last minute
# This example assumes log lines include timestamps that can be parsed.
# For simplicity, we'll tail the last N lines and approximate the window.
# In a robust system, you'd use a log processing tool or persistent state
# to track the last read position and precise timestamps.
# For this example, let's assume `journalctl` with a specific systemd unit.
# If using plain file logs, `grep` and `date` parsing might be heavier.

# Using journalctl for a systemd service, which is more robust for recent logs:
# This assumes auth-service is a systemd unit, e.g., auth-service.service
# If not, adjust to `grep` from `LOG_FILE` after ensuring log timestamp parsing.
ERROR_COUNT=$(journalctl -u auth-service.service --since="1 minute ago" | grep -c
```

```
"AuthenticationFailed")

# If using direct file tail, might look like this (less precise without proper timestamp
parsing):
# ERROR_COUNT=$(tail -n 1000 "$LOG_FILE" | grep -c "AuthenticationFailed") # Tail last 1000
lines as a rough window

# Create temporary metric file
cat <<EOF > "$TEMP_METRIC_FILE"
# HELP ${METRIC_NAME} Total number of authentication failed errors in auth-service.
# TYPE ${METRIC_NAME} counter
${METRIC_NAME}{app="${APP_NAME}"} ${ERROR_COUNT}
EOF

# Atomically move the temporary file to the final destination
mv "$TEMP_METRIC_FILE" "$FINAL_METRIC_FILE"

echo "Metrics updated for $APP_NAME. Error count: $ERROR_COUNT"

# Schedule this script with cron to run every minute:
# `crontab -e`
# * * * * * /usr/local/bin/collect_auth_errors.sh
```

This script would run every minute, update the Prometheus metric, and Prometheus would then evaluate the alerting rule based on the collected count. This provides a lightweight, flexible way to integrate application-specific log patterns into your monitoring system.

Q43. Design a robust Bash script to serve as a health check for a critical web application endpoint, considering scenarios like network issues, application slowness, and incorrect HTTP status codes. How would this integrate into a Kubernetes LivenessProbe?

Detailed Answer:

A robust Bash health check script for a web application needs to go beyond a simple `curl` to check for a 200 OK status. It should consider:

1. Network Reachability: Can the target host/port be reached? (e.g., `nc -zvw3`).

2. HTTP Status Code: Is the application returning the expected success code (e.g., 200, 204)?

3. Response Content: Does the response body contain a specific string indicating application health? (e.g., "STATUS: UP", "HEALTHY"). This guards against cases where an application returns 200 OK but is serving a static error page.

4. Response Time: Is the application responding within an acceptable latency? Slow responses can be as bad as errors.

5. Retries and Timeouts: Brief network glitches shouldn't immediately fail the check. Retries with delays are crucial. Timeouts prevent hanging.

The script should return an exit code of `0` for success and a non-zero exit code for failure, which is what Kubernetes LivenessProbes expect.

Integration into Kubernetes LivenessProbe:

Kubernetes supports `exec` probes, where it executes a command inside the container. If the command exits with status 0, the probe succeeds; otherwise, it fails. A persistent failure (configured by `failureThreshold`) leads to the pod being restarted.

The LivenessProbe configuration in a Kubernetes Deployment/Pod manifest would look like this:

```
livenessProbe:
  exec:
    command: ["/bin/bash", "-c", "/usr/local/bin/healthcheck.sh
http://localhost:8080/health"]
  initialDelaySeconds: 15
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3
```

Here, `initialDelaySeconds` gives the app time to start, `periodSeconds` defines how often the check runs, `timeoutSeconds` is the maximum time the script has to execute, and `failureThreshold` determines how many consecutive failures trigger a restart.

Production Scenario / Practical Example:

Let's create a script `/usr/local/bin/healthcheck.sh` inside our application container to check `http://localhost:8080/health`.

```
#!/bin/bash
# Script: /usr/local/bin/healthcheck.sh
# Arguments: <URL_TO_CHECK> [<EXPECTED_STATUS_CODE>] [<EXPECTED_RESPONSE_STRING>]

URL="${1:-http://localhost:8080/health}"
EXPECTED_STATUS="${2:-200}"
EXPECTED_STRING="${3:-"healthy"}" # Case-insensitive check for "healthy" in response

MAX_RETRIES=3
RETRY_DELAY=2 # seconds
TIMEOUT=4     # seconds for curl
CURL_OPTIONS="-s -o /dev/null -w %{http_code}:::%{time_total}:::%{json}" # Silent, output to
null, get status and time

echo "Starting health check for $URL (Expected Status: $EXPECTED_STATUS, Expected String:
'$EXPECTED_STRING')"

for i in $(seq 1 $MAX_RETRIES); do
    echo "Attempt $i/$MAX_RETRIES..."
```

```
    # Use curl to get HTTP status code and total time.
    # Also capture response body to check for specific string.
    # Using a temporary file for the body to avoid race conditions with stdout.
    RESPONSE_BODY_FILE=$(mktemp)

    CURL_OUTPUT=$(curl -m "$TIMEOUT" -s -w "%{http_code}:::%{time_total}" -o
"$RESPONSE_BODY_FILE" "$URL")
    CURL_STATUS=$? # Exit status of curl

    if [ "$CURL_STATUS" -ne 0 ]; then
        echo "CURL ERROR: Command failed with exit status $CURL_STATUS."
        if [ "$CURL_STATUS" -eq 28 ]; then
            echo "Timeout reached after $TIMEOUT seconds."
        fi
        rm -f "$RESPONSE_BODY_FILE"
        sleep "$RETRY_DELAY"
        continue
    fi

    HTTP_CODE=$(echo "$CURL_OUTPUT" | cut -d':' -f1)
    TOTAL_TIME=$(echo "$CURL_OUTPUT" | cut -d':' -f2)

    echo "  HTTP Status: $HTTP_CODE, Response Time: ${TOTAL_TIME}s"

    # Check HTTP status code
    if [ "$HTTP_CODE" -ne "$EXPECTED_STATUS" ]; then
        echo "  FAILURE: Unexpected HTTP status code $HTTP_CODE (expected $EXPECTED_STATUS)."
        rm -f "$RESPONSE_BODY_FILE"
        sleep "$RETRY_DELAY"
        continue
    fi

    # Check response body content
    if ! grep -qi "$EXPECTED_STRING" "$RESPONSE_BODY_FILE"; then
        echo "  FAILURE: Expected string '$EXPECTED_STRING' not found in response body."
        # cat "$RESPONSE_BODY_FILE" # Uncomment for debugging response body
        rm -f "$RESPONSE_BODY_FILE"
        sleep "$RETRY_DELAY"
        continue
    fi

    # Check response time for slowness (e.g., > 1 second)
    # Using `bc` for floating point comparison
    if (( $(echo "$TOTAL_TIME > 1.0" | bc -l) )); then
        echo "  WARNING: Application response is slow ($TOTAL_TIME s)."
        # Depending on criticality, this might be a soft failure or just a warning.
        # For a LivenessProbe, we want hard failures. Let's make it fail if too slow.
        # If this check needs to be optional, add another argument.
```

```
        # For now, if it's slow, we consider it a failure for immediate restart.
        echo "  FAILURE: Response time ($TOTAL_TIME s) exceeded threshold (1.0s)."
        rm -f "$RESPONSE_BODY_FILE"
        sleep "$RETRY_DELAY"
        continue
    fi

    echo "Health check PASSED."
    rm -f "$RESPONSE_BODY_FILE"
    exit 0 # Success!
done

echo "Health check FAILED after $MAX_RETRIES attempts."
rm -f "$RESPONSE_BODY_FILE" # Ensure cleanup on final failure
exit 1 # Failure!
```

This script is placed inside the container, e.g., via a `COPY` command in the Dockerfile. It provides granular checks and robust retries, ensuring that transient issues don't lead to unnecessary restarts, while genuine problems are caught and addressed.

Q44. You need to create custom Prometheus metrics from a legacy system that only provides its status via a simple text file (`/opt/app/status.txt`) containing key-value pairs. How would you use Bash to parse this file and expose these metrics using the Node Exporter Textfile Collector, handling different data types (counters, gauges)?

Detailed Answer:

The Prometheus Node Exporter's Textfile Collector is perfect for integrating custom metrics from systems that don't natively expose a Prometheus endpoint. The collector simply reads files from a designated directory (e.g., `/var/lib/node_exporter/textfile_collector`) and exposes their content as metrics. The key is to generate these files in the correct Prometheus exposition format.

The Bash script would involve:

1. Reading the Status File: Parse `/opt/app/status.txt` line by line.

2. Extracting Key-Value Pairs: Use `awk` or `grep` and `cut` to extract the metric name and its value from each line. Assume a simple `KEY=VALUE` format.

3. Determining Metric Type: Based on conventions or explicit configuration, decide if a metric is a `gauge` (can go up and down, e.g., `current_connections`) or a `counter` (only ever increases, e.g., `total_requests_processed`).

4. Formatting Metrics: Convert the extracted data into the Prometheus text exposition format, including `HELP` and `TYPE` lines.

- `# HELP <metric_name> <description>`
- `# TYPE <metric_name> <gauge|counter|summary|histogram>`
- `<metric_name>{<label_name>="<label_value>",...} <value>`

5. Atomic Update: Write the generated metrics to a temporary file, then atomically move it to the Node Exporter's configured textfile collection directory. This ensures Prometheus always reads a complete and consistent set of metrics.

6. Scheduling: Schedule the script via `cron` or `systemd.timer` to run at a suitable interval (e.g., every 30-60 seconds), depending on the desired metric freshness.

Production Scenario / Practical Example:

Assume `/opt/app/status.txt` looks like this:

```
app_version=1.2.3
current_connections=45
total_requests_processed=123456
last_restart_timestamp=1678886400
is_active=1
```

We want `current_connections` to be a gauge, `total_requests_processed` a counter, `app_version` and `last_restart_timestamp` as labels, and `is_active` as a boolean gauge.

```
#!/bin/bash
# Script: /usr/local/bin/legacy_app_metrics.sh

STATUS_FILE="/opt/app/status.txt"
METRICS_DIR="/var/lib/node_exporter/textfile_collector"
TEMP_METRIC_FILE="${METRICS_DIR}/legacy_app_status.prom.tmp"
FINAL_METRIC_FILE="${METRICS_DIR}/legacy_app_status.prom"

# Ensure the metrics directory exists
mkdir -p "$METRICS_DIR"

if [ ! -f "$STATUS_FILE" ]; then
    echo "ERROR: Status file $STATUS_FILE not found." >&2
    exit 1
fi

# Initialize metric content with a header
echo "# This file is generated by legacy_app_metrics.sh" > "$TEMP_METRIC_FILE"

# Read the status file and parse
declare -A APP_STATUS
while IFS='=' read -r key value; do
    # Trim whitespace from key and value
    key=$(echo "$key" | xargs)
    value=$(echo "$value" | xargs)
    APP_STATUS["$key"]="$value"
done < "$STATUS_FILE"

# Extract common labels (e.g., app_version)
```

```
APP_VERSION="${APP_STATUS[app_version]:-unknown}"
LAST_RESTART_TIMESTAMP="${APP_STATUS[last_restart_timestamp]:-0}"
COMMON_LABELS="app_version=\"$APP_VERSION\",last_restart_timestamp=\"$LAST_RESTART_TIMESTAMP\
""

# --- current_connections (Gauge) ---
CURRENT_CONNECTIONS="${APP_STATUS[current_connections]:-0}"
cat <<EOF >> "$TEMP_METRIC_FILE"
# HELP legacy_app_current_connections Current number of active connections to the legacy
application.
# TYPE legacy_app_current_connections gauge
legacy_app_current_connections{$COMMON_LABELS} $CURRENT_CONNECTIONS
EOF

# --- total_requests_processed (Counter) ---
TOTAL_REQUESTS="${APP_STATUS[total_requests_processed]:-0}"
cat <<EOF >> "$TEMP_METRIC_FILE"
# HELP legacy_app_total_requests_processed Total number of requests processed by the legacy
application.
# TYPE legacy_app_total_requests_processed counter
legacy_app_total_requests_processed{$COMMON_LABELS} $TOTAL_REQUESTS
EOF

# --- is_active (Gauge, 0 or 1) ---
IS_ACTIVE="${APP_STATUS[is_active]:-0}"
cat <<EOF >> "$TEMP_METRIC_FILE"
# HELP legacy_app_is_active Is the legacy application currently active (1=yes, 0=no).
# TYPE legacy_app_is_active gauge
legacy_app_is_active{$COMMON_LABELS} $IS_ACTIVE
EOF

# Atomically move the temporary file to the final destination
mv "$TEMP_METRIC_FILE" "$FINAL_METRIC_FILE"

echo "Legacy app metrics updated at $(date)"

# Schedule this script with cron to run every minute:
# `crontab -e`
# * * * * * /usr/local/bin/legacy_app_metrics.sh
```

This script efficiently parses the text file, formats the metrics with appropriate types and labels, and makes them available for Prometheus scraping. This method allows you to bring metrics from almost any legacy system into your modern monitoring stack.

Q45. How would you design a Bash-based incident response runbook script that automates the collection of critical diagnostic information (logs, system metrics, process states) from multiple services/hosts and provides actionable insights during a production incident?

Detailed Answer:

A Bash-based incident response runbook script needs to be robust, idempotent, and provide clear, structured output. Its primary goal is to rapidly gather disparate pieces of information that an SRE would manually collect during an incident, consolidate them, and potentially perform initial analysis or provide pointers.

Key design principles:

1. Modularity: Break down data collection into functions (e.g., `collect_system_metrics`, `collect_logs`, `collect_process_info`). This allows for easy extension and debugging.

2. Target Scope: The script should be able to operate on a single host, a list of hosts, or a Kubernetes cluster. This implies using SSH for remote execution or `kubectl` for cluster resources.

3. Non-disruptive: Diagnostic commands should have minimal impact on the running system. Avoid commands that write extensively or consume significant CPU/memory.

4. Time-bound: Each collection step should have a timeout to prevent the script from hanging on an unresponsive system.

5. Output Management: Collected data should be stored in a well-organized directory structure (e.g., `incident_data/<timestamp>/<hostname>/<category>`) on a central location or a shared volume. Use `mktemp` for temporary files.

6. Error Handling: Log errors but continue execution where possible to gather maximum information. Use `set -e` carefully, or wrap critical sections in subshells that check exit codes.

7. Summary/Analysis: After collection, the script could perform basic `grep` or `awk` operations to highlight anomalies or common error patterns, providing immediate insights.

8. Security: Avoid hardcoding credentials. Use SSH keys with agents, or vault-managed secrets.

The script would typically accept parameters like `incident_id`, `target_hosts`, `target_services`.

Production Scenario / Practical Example:

Let's imagine an incident affecting `web-service` and `db-service` running on VMs `web-01.example.com` and `db-01.example.com` respectively. We need to collect system metrics, service logs, and process states.

```
#!/bin/bash
# Script: /usr/local/bin/incident_diag_collector.sh

INCIDENT_ID="${1:-$(date +%Y%m%d%H%M%S)}" # Default to timestamp if no ID provided
TARGET_HOSTS="${2:-"web-01.example.com db-01.example.com"}" # Space-separated list
OUTPUT_BASE_DIR="/var/log/incident_diagnostics"
COLLECTION_DIR="${OUTPUT_BASE_DIR}/${INCIDENT_ID}"

# Set robust error handling
set -euo pipefail

# --- Helper functions ---

log_message() {
```

```
    echo "$(date +'%Y-%m-%d %H:%M:%S') - $1" | tee -a "${COLLECTION_DIR}/script_log.txt"
}

run_remote_command() {
    local host="$1"
    local output_file="$2"
    local command="$3"
    local timeout_sec="${4:-10}" # Default timeout 10 seconds

    log_message "  -> Running command on $host: $command"

    # Using 'ssh -o BatchMode=yes -o ConnectTimeout=5' for non-interactive and quick connect
fail
    # Use 'timeout' for the command itself
    if ! timeout "$timeout_sec" ssh -o BatchMode=yes -o ConnectTimeout=5 "$host" "$command" >
"$output_file" 2>&1; then
        log_message "  -> WARNING: Command failed or timed out on $host. Output in
$output_file"
        echo "Command: $command" >> "$output_file"
        echo "Exit status: $?" >> "$output_file"
    fi
}

# --- Collection functions ---

collect_system_metrics() {
    local host="$1"
    local host_dir="${COLLECTION_DIR}/${host}"
    mkdir -p "$host_dir/system_metrics"
    log_message "Collecting system metrics for $host..."

    run_remote_command "$host" "$host_dir/system_metrics/uptime.txt" "uptime"
    run_remote_command "$host" "$host_dir/system_metrics/free_m.txt" "free -m"
    run_remote_command "$host" "$host_dir/system_metrics/df_h.txt" "df -h"
    run_remote_command "$host" "$host_dir/system_metrics/top_b_n1.txt" "top -b -n 1" 20 #
Give more time for top
    run_remote_command "$host" "$host_dir/system_metrics/netstat_tulnp.txt" "netstat -tulnp"
    run_remote_command "$host" "$host_dir/system_metrics/vmstat_1_5.txt" "vmstat 1 5" 10 # 5
samples over 5 seconds
    run_remote_command "$host" "$host_dir/system_metrics/iostat_x_1_5.txt" "iostat -x 1 5" 10
# 5 samples over 5 seconds
}

collect_service_logs() {
    local host="$1"
    local service_name="$2"
    local host_dir="${COLLECTION_DIR}/${host}"
    mkdir -p "$host_dir/logs"
```

```
    log_message "Collecting logs for $service_name on $host..."

    # Assume systemd service for journalctl
    run_remote_command "$host" "$host_dir/logs/${service_name}_journal_last_hour.log"
"journalctl -u $service_name --since '1 hour ago' --no-pager" 60 # Give more time for logs
    # If using plain file logs:
    # run_remote_command "$host" "$host_dir/logs/${service_name}_access_tail.log" "tail -n
1000 /var/log/${service_name}/access.log"
    # run_remote_command "$host" "$host_dir/logs/${service_name}_error_tail.log" "tail -n
1000 /var/log/${service_name}/error.log"
}

collect_process_info() {
    local host="$1"
    local host_dir="${COLLECTION_DIR}/${host}"
    mkdir -p "$host_dir/processes"
    log_message "Collecting process information for $host..."

    run_remote_command "$host" "$host_dir/processes/ps_auxf.txt" "ps auxf"
    run_remote_command "$host" "$host_dir/processes/lsof_i.txt" "lsof -i"
    run_remote_command "$host" "$host_dir/processes/strace_top_pids.txt" "ps -eo
pid,pcpu,comm --sort=-pcpu | head -n 5 | awk 'NR>1 {print \$1}' | xargs -r -I{} strace -p {}
-c -o /dev/null -T -q -D -s 1024 -r -f -t -tt -v" 30 # Strace top 5 PIDs for 30s
}

# --- Main execution ---

log_message "Starting incident diagnostic collection for incident $INCIDENT_ID"
mkdir -p "$COLLECTION_DIR"

for host in $TARGET_HOSTS; do
    log_message "Processing host: $host"
    collect_system_metrics "$host" &
    collect_service_logs "$host" "web-service" & # Adjust service name per host
    collect_service_logs "$host" "db-service" & # Adjust service name per host
    collect_process_info "$host" &
done

wait # Wait for all background collection jobs to finish

log_message "--- Initial Analysis / Summary ---"
# Example: Check for "ERROR" or "Exception" in collected logs
for host in $TARGET_HOSTS; do
    log_message "Host: $host - Log Error Summary:"
    if grep -qE "ERROR|Exception|Failed" "${COLLECTION_DIR}/${host}/logs/"*.log 2>/dev/null;
then
        log_message "  Potential errors found. Sample:"
        grep -E "ERROR|Exception|Failed" "${COLLECTION_DIR}/${host}/logs/"*.log | head -n 10
```

```
| tee -a "${COLLECTION_DIR}/script_log.txt"
    else
        log_message "  No significant errors (ERROR|Exception|Failed) found in recent logs."
    fi

    log_message "Host: $host - Top CPU processes (from top output):"
    if [ -f "${COLLECTION_DIR}/${host}/system_metrics/top_b_n1.txt" ]; then
        grep -A 5 "PID USER" "${COLLECTION_DIR}/${host}/system_metrics/top_b_n1.txt" | tee -a
"${COLLECTION_DIR}/script_log.txt"
    fi
done

log_message "Diagnostic collection complete. Data stored in $COLLECTION_DIR"
log_message "Please review the collected files for detailed analysis."
```

This script would be executed from a central jump host. It parallelizes collection per host using `&` and `wait`, providing a faster data gathering process. The output is structured and can be easily compressed and archived for later RCA. The initial analysis provides immediate pointers to common issues, making it a powerful tool in an SRE's arsenal.

Q46. When writing Bash scripts for production SRE tasks, how do you ensure robustness, safety, and atomicity? Discuss specific techniques like error handling, idempotency, resource cleanup, and locking mechanisms.

Detailed Answer:

Ensuring robustness, safety, and atomicity in production Bash scripts is paramount to prevent outages, data corruption, or inconsistent states. These scripts often operate on critical systems, so a failure or partial execution can have severe consequences.

1. Error Handling (`set -euxo pipefail` and `trap`):
   - `set -e`: Exit immediately if a command exits with a non-zero status. This prevents scripts from continuing with an invalid state.
   - `set -u`: Treat unset variables as an error. Catches typos and ensures variables are initialized.
   - `set -o pipefail`: The exit status of a pipeline is the status of the last command to exit with a non-zero status, or zero if all commands exited successfully. This is crucial for pipelines like `grep ... | awk ...`.
   - `set -x`: Print commands and their arguments as they are executed (useful for debugging, but be cautious with sensitive data).
   - `trap 'command' ERR EXIT INT TERM`: Execute a command or function on specific signals.
   - `ERR`: Executes on any command exit with a non-zero status (if `set -e` is not active, or if you need to catch specific errors).
   - `EXIT`: Always executes when the script exits, regardless of success or failure. Ideal for cleanup.
   - `INT`, `TERM`: Handle Ctrl+C (interrupt) and termination signals gracefully.
2. Idempotency:

An idempotent script produces the same result regardless of how many times it's executed with the same inputs. This is vital for automation, retries, and scheduled tasks.

- **Conditional Execution**: Check if a resource or state already exists before creating/modifying it (e.g., `mkdir -p`, `if [ ! -f /path/to/file ]; then ... fi`).
- **State Tracking**: Maintain a persistent state (e.g., a simple flag file or a database entry) to record successful completion of phases.
- **Compare and Update**: Instead of blindly overwriting, compare current state with desired state and apply only necessary changes (e.g., `rsync --checksum --update`).

3. Resource Cleanup (`trap EXIT`):

Ensure temporary files, directories, and other transient resources are always cleaned up, even if the script fails or is interrupted.

- Use `mktemp -d` for temporary directories and `mktemp` for temporary files, which generate unique names.
- Register cleanup actions with `trap EXIT`. A common pattern is to define a `cleanup` function and then `trap cleanup EXIT`.

4. Locking Mechanisms:

Prevent multiple instances of the same script from running concurrently, which can lead to race conditions, resource contention, or data corruption.

- **File-based Locking (`flock`)**: The most common and robust Bash-native locking mechanism. `flock` provides exclusive access to a file, blocking other processes until the lock is released or a timeout occurs.

Production Scenario / Practical Example:

Here's a script snippet demonstrating these techniques, performing a hypothetical configuration update:

```
#!/bin/bash
# Script: /usr/local/bin/safe_config_updater.sh

# Configuration
CONFIG_DIR="/etc/my_app"
NEW_CONFIG_SOURCE="/tmp/new_config" # Assume this file is provided or generated
LOCK_FILE="/var/lock/my_app_config_updater.lock"
BACKUP_DIR="/var/backups/my_app_config"
LOG_FILE="/var/log/my_app_config_updater.log"

# --- Robustness: Error Handling (set -euxo pipefail) ---
# e: Exit immediately if a command exits with a non-zero status.
# u: Treat unset variables as an error.
# x: Print commands and their arguments as they are executed (for debugging).
# o pipefail: The exit status of a pipeline is the status of the last command to exit with a
non-zero status.
set -euxo pipefail

# --- Robustness: Resource Cleanup (trap EXIT) ---
```

```
# Create a cleanup function
cleanup() {
    local exit_code=$? # Capture the exit code of the script
    echo "$(date) - Script finished with exit code $exit_code. Cleaning up temporary
resources." >> "$LOG_FILE"
    rm -rf "$TMP_CONFIG_COPY" # Remove temporary directory if it exists
    rm -f "$LOCK_FILE" # Ensure lock file is released on exit

    if [ "$exit_code" -ne 0 ]; then
        echo "$(date) - ERROR: Script failed. Investigate logs in $LOG_FILE." >> "$LOG_FILE"
        # Potentially send an alert here
    fi
}
# Trap the EXIT signal to ensure cleanup always runs
trap cleanup EXIT INT TERM

echo "$(date) - Starting configuration update process." >> "$LOG_FILE"

# --- Safety: Locking Mechanism (flock) ---
# Acquire an exclusive lock on the lock file. If another instance is running,
# this command will block until the lock is released or it times out (-w).
# Using `exec 200>` associates file descriptor 200 with the lock file.
(
    flock -xn 200 || { echo "$(date) - ERROR: Another instance of config updater is running.
Exiting." >> "$LOG_FILE"; exit 1; }

    echo "$(date) - Lock acquired." >> "$LOG_FILE"

    # --- Robustness: Temporary Directory for Atomic Operations ---
    # Use mktemp -d to create a unique temporary directory
    TMP_CONFIG_COPY=$(mktemp -d "/tmp/my_app_config.XXXXXX")
    echo "$(date) - Created temporary directory: $TMP_CONFIG_COPY" >> "$LOG_FILE"

    # --- Safety: Idempotency & Backup ---
    # 1. Create backup before any changes
    mkdir -p "$BACKUP_DIR"
    cp -r "$CONFIG_DIR" "${BACKUP_DIR}/my_app_config_$(date +%Y%m%d%H%M%S)"
    echo "$(date) - Configuration backup created." >> "$LOG_FILE"

    # 2. Copy new config to temporary location
    cp "$NEW_CONFIG_SOURCE" "$TMP_CONFIG_COPY/application.conf"
    echo "$(date) - New configuration copied to temporary location." >> "$LOG_FILE"

    # 3. Validate new configuration (hypothetical example)
    if ! grep -q "required_setting=" "$TMP_CONFIG_COPY/application.conf"; then
        echo "$(date) - ERROR: New configuration is invalid (missing 'required_setting').
Aborting." >> "$LOG_FILE"
        exit 1 # Trigger trap EXIT
```

```
    fi
    echo "$(date) - New configuration validated." >> "$LOG_FILE"

    # 4. Atomically replace old config with new config
    # Use rsync for atomic replacement; it first copies to a temp file then renames.
    # mv is also atomic on a single filesystem.
    rsync -av --delete "$TMP_CONFIG_COPY/application.conf" "$CONFIG_DIR/"
    echo "$(date) - Configuration updated atomically." >> "$LOG_FILE"

    # 5. Idempotent service restart (if needed)
    # Check if the service needs a restart (e.g., config diff)
    if systemctl is-active --quiet my-app-service; then
        echo "$(date) - Restarting my-app-service for new configuration to take effect." >>
"$LOG_FILE"
        systemctl restart my-app-service
    else
        echo "$(date) - my-app-service is not active, skipping restart." >> "$LOG_FILE"
    fi

    echo "$(date) - Configuration update successful." >> "$LOG_FILE"

) 200>"$LOCK_FILE" # Associate FD 200 with the lock file for flock
```

This script combines several techniques for robust and safe execution. The `flock` ensures only one instance runs, `trap cleanup EXIT` guarantees temporary files are removed, `set -euxo pipefail` catches errors early, and atomic file operations (like `rsync` or `mv`) combined with backups contribute to a production-grade script.

Q47. You need to automate the provisioning of a cloud resource (e.g., an S3 bucket or a VM instance) using a cloud provider's CLI (like AWS CLI, Azure CLI, gcloud) from a Bash script. How do you handle authentication, error handling, and parameterization effectively?

Detailed Answer:

Automating cloud resource provisioning with Bash and CLI tools is a common SRE task. Effective handling of authentication, error management, and parameterization is crucial for security, reliability, and reusability.

1. Authentication:

    - **Environment Variables**: Cloud CLIs often respect environment variables (e.g., `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`). This is suitable for CI/CD environments where credentials can be securely injected.

    - **IAM Roles/Service Principals**: For scripts running on cloud VMs or within Kubernetes clusters, leverage IAM roles (AWS), Managed Identities (Azure), or Service Accounts (GCP). The CLI automatically picks up credentials from the instance metadata service, eliminating the need to store credentials in the script or environment. This is the **most secure and recommended approach**.

    - **Configuration Files**: CLIs can read credentials from `~/.aws/credentials`, `~/.azure/credentials`, etc. While convenient for interactive use, avoid committing these files or relying on them in automated pipelines

without extreme care.

- **Temporary Credentials**: For specific, short-lived tasks, CLIs can generate temporary credentials, often in combination with MFA or federated identity.

2. Error Handling:

- **Exit Status**: Cloud CLI commands return non-zero exit codes on failure. Use `set -e` or explicitly check `$?` after each critical command.
- **JSON Output Parsing**: Cloud CLIs usually output JSON. Use `jq` to parse the output and check for specific error messages or the absence of expected values. This allows for more granular error detection than just exit codes.
- **Logging**: Redirect stdout and stderr to a log file. Include timestamps and relevant context.
- **Retry Mechanisms**: For transient network issues or API rate limiting, implement exponential backoff and retry logic (e.g., using `aws-cli-plugin-retry` or a custom Bash loop with `sleep`).

3. Parameterization:

- **Command-line Arguments**: Use `$1`, `$2`, etc., or `getopts`/`getopt` for more complex options, to pass dynamic values (e.g., bucket name, region, instance type).
- **Configuration Files**: For complex or static configurations, use a separate YAML or JSON file. Bash can parse these using `yq` or `jq`.
- **Environment Variables**: Can be used for common parameters that might change between environments (e.g., `ENV=prod`, `ENV=dev`).
- **Default Values**: Provide sensible defaults for optional parameters to improve script usability.

Production Scenario / Practical Example:

Let's create an S3 bucket in AWS, parameterized by name and region. The script will assume an IAM role is attached to the host for authentication.

```
#!/bin/bash
# Script: /usr/local/bin/provision_s3_bucket.sh

# --- Parameterization ---
BUCKET_NAME="${1:-}" # Required: Bucket name
REGION="${2:-us-east-1}" # Optional: AWS region, default to us-east-1
PROFILE="${3:-}" # Optional: AWS profile for local testing, empty for IAM role

# --- Environment Configuration ---
LOG_FILE="/var/log/provision_s3_bucket.log"
# Set AWS CLI output format to JSON for easier parsing
export AWS_DEFAULT_OUTPUT="json"

# --- Error Handling (set -euxo pipefail) ---
set -euo pipefail

# --- Logging function ---
log_message() {
```

```
    echo "$(date +'%Y-%m-%d %H:%M:%S') - $1" | tee -a "$LOG_FILE"
}

# --- Cleanup function (if needed, e.g., for temporary files) ---
cleanup() {
    local exit_code=$?
    log_message "Script finished with exit code $exit_code."
    # No temporary files to clean up in this specific example
}
trap cleanup EXIT INT TERM

# --- Input Validation ---
if [ -z "$BUCKET_NAME" ]; then
    log_message "ERROR: Bucket name is required. Usage: $0 <BUCKET_NAME> [REGION] [PROFILE]"
    exit 1
fi

log_message "Starting S3 bucket provisioning for bucket: $BUCKET_NAME in region: $REGION"

# Construct AWS CLI options dynamically based on PROFILE
AWS_CLI_OPTS=""
if [ -n "$PROFILE" ]; then
    AWS_CLI_OPTS="--profile $PROFILE"
    log_message "Using AWS profile: $PROFILE"
else
    log_message "Using IAM role for authentication (no profile specified)."
fi

# --- Idempotency Check: Check if bucket already exists ---
log_message "Checking if bucket '$BUCKET_NAME' already exists..."
if aws $AWS_CLI_OPTS s3api head-bucket --bucket "$BUCKET_NAME" --region "$REGION" >/dev/null
2>&1; then
    log_message "Bucket '$BUCKET_NAME' already exists. Skipping creation."
else
    # --- Provisioning Step ---
    log_message "Bucket '$BUCKET_NAME' does not exist. Creating..."
    CREATE_OUTPUT=$(aws $AWS_CLI_OPTS s3api create-bucket \
        --bucket "$BUCKET_NAME" \
        --region "$REGION" \
        --create-bucket-configuration LocationConstraint="$REGION")

    # --- Error Handling: Check AWS CLI output and exit status ---
    if [ $? -ne 0 ]; then
        log_message "ERROR: Failed to create bucket '$BUCKET_NAME'. Details: $CREATE_OUTPUT"
        exit 1
    fi
    log_message "Bucket creation successful: $(echo "$CREATE_OUTPUT" | jq -r '.Location')"
fi
```

```
# --- Additional configuration (Idempotent) ---
# Example: Set public access block
log_message "Configuring public access block for bucket '$BUCKET_NAME'..."
BLOCK_PUBLIC_OUTPUT=$(aws $AWS_CLI_OPTS s3api put-public-access-block \
    --bucket "$BUCKET_NAME" \
    --public-access-block-configuration
"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
" \
    --region "$REGION")
if [ $? -ne 0 ]; then
    log_message "ERROR: Failed to configure public access block. Details:
$BLOCK_PUBLIC_OUTPUT"
    # Decide if this is a fatal error or a warning. For security, likely fatal.
    exit 1
fi
log_message "Public access block configured successfully."

log_message "S3 bucket '$BUCKET_NAME' provisioning complete."
```

This script demonstrates robust error handling, input validation, and idempotency. By relying on IAM roles, it avoids hardcoding credentials. `jq` could be further used to extract specific attributes from the JSON output for logging or subsequent actions.

Q48. How would you use Bash scripting to diagnose and troubleshoot intermittent network connectivity issues between two servers (e.g., a web server and a database server) in a production environment? Detail the commands and logic you'd employ.

Detailed Answer:

Diagnosing intermittent network connectivity issues requires collecting data over time to capture the "intermittent" nature. A Bash script can automate this repetitive data collection and highlight patterns or specific points of failure. The approach involves collecting information at various network layers.

Commands and Logic:

1. Basic Reachability (`ping`, `nc`):
   - `ping -c 5 <target_ip>`: Check basic ICMP reachability and latency.
   - `nc -zv <target_ip> <target_port>`: Check if a specific port is open and reachable (e.g., DB port).
   - Loop these checks every few seconds or minutes to catch intermittent drops.
2. Route and Interface Status (`ip route`, `ip addr`, `ethtool`):
   - `ip route show`: Verify routing tables on both source and target.
   - `ip addr show`: Check interface status, IP addresses, and errors.
   - `ethtool <interface_name>`: Check link status, speed, duplex, and error counters (rx/tx errors, collisions) on network interfaces.

3. Connection States and Statistics (`ss`, `netstat`):
   - `ss -s`: Summarize socket statistics (total connections, listen queues).
   - `ss -ntup | grep <target_port>`: List active TCP connections, states (ESTABLISHED, SYN_SENT, TIME_WAIT), and associated processes. This helps identify connections stuck in non-ESTABLISHED states.
   - `netstat -i`: Show interface statistics, including packet errors and drops.
4. Packet Tracing (`traceroute`, `mtr`, `tcpdump`):
   - `traceroute -n <target_ip>` or `mtr -c 10 <target_ip>`: Trace the network path and identify specific hops with high latency or packet loss. `mtr` is superior for continuous monitoring.
   - `tcpdump -i <interface> -nn -s0 -w <file.pcap> host <target_ip> and port <target_port>`: Capture raw packets on both ends during the intermittent failure. This is invaluable for deep analysis (e.g., using Wireshark) to see retransmissions, dropped packets, or unexpected resets.
5. DNS Resolution (`dig`, `host`):
   - If connecting by hostname, `dig <hostname>` or `host <hostname>` on both servers to ensure consistent DNS resolution.

Script Logic:

The script would encapsulate these checks in a loop, capturing output with timestamps, and performing some initial analysis.

Production Scenario / Practical Example:

We are troubleshooting intermittent connectivity between a web server (`web-server-01`, IP: `10.0.0.10`) and a database server (`db-server-01`, IP: `10.0.0.20`, Port: `5432`).

```
#!/bin/bash
# Script: /usr/local/bin/network_troubleshooter.sh

TARGET_DB_IP="10.0.0.20"
TARGET_DB_PORT="5432"
# Assuming this script runs on web-server-01
LOCAL_WEB_IP="10.0.0.10" # For reference in output
LOG_DIR="/var/log/net_diag_$(date +%Y%m%d%H%M%S)"
DURATION=600 # Run for 10 minutes (600 seconds)
INTERVAL=5   # Run checks every 5 seconds
PACKET_CAPTURE_DURATION=30 # Capture packets for 30 seconds if an issue is detected

mkdir -p "$LOG_DIR"
exec > >(tee -a "$LOG_DIR/overall_output.log") 2>&1 # Redirect all output to log file and
stdout

log_message() {
    echo "$(date +'%Y-%m-%d %H:%M:%S') - $1"
}
```

```
collect_basic_info() {
    log_message "Collecting initial system network information..."
    echo "--- Host: $(hostname) IP: $LOCAL_WEB_IP ---"
    echo "--- ip addr show ---" && ip addr show
    echo "--- ip route show ---" && ip route show
    echo "--- ethtool eth0 ---" && ethtool eth0 || true # Assuming eth0, ignore error if not
found
    echo "--- DNS Resolution for $TARGET_DB_IP (reverse) ---" && host "$TARGET_DB_IP"
    echo "--- DNS Resolution for db-server-01 (forward) ---" && dig +short
db-server-01.example.com
    echo "--- ss -s (socket summary) ---" && ss -s
    echo "--- netstat -i (interface stats) ---" && netstat -i
}

run_checks_loop() {
    log_message "Starting intermittent connectivity checks for $DURATION seconds (every
$INTERVAL sec)..."
    local start_time=$(date +%s)
    local end_time=$((start_time + DURATION))
    local check_count=0

    while [ "$(date +%s)" -lt "$end_time" ]; do
        check_count=$((check_count + 1))
        log_message "--- Check $check_count at $(date) ---"

        # Ping check
        PING_RESULT=$(ping -c 3 -W 1 "$TARGET_DB_IP") # 3 packets, 1 sec timeout per packet
        PING_EXIT_CODE=$?
        echo "$PING_RESULT"
        if [ "$PING_EXIT_CODE" -ne 0 ]; then
            log_message "WARNING: Ping to $TARGET_DB_IP failed or had packet loss!"
            TRIGGER_PACKET_CAPTURE=true
        fi

        # Netcat port check
        if ! nc -zv -w 2 "$TARGET_DB_IP" "$TARGET_DB_PORT"; then # Timeout 2 seconds
            log_message "WARNING: Port $TARGET_DB_PORT on $TARGET_DB_IP is not reachable!"
            TRIGGER_PACKET_CAPTURE=true
        fi

        # Traceroute/MTR (only if issues detected or periodically for path changes)
        if (( check_count % 10 == 0 )) || [ "$PING_EXIT_CODE" -ne 0 ] || [
"$TRIGGER_PACKET_CAPTURE" = true ]; then
            log_message "Running MTR to $TARGET_DB_IP..."
            mtr -c 5 -n "$TARGET_DB_IP"
        fi

        # TCP Connection States for DB port
```

```
        log_message "DB connection states (ss):"
        ss -ntup | grep ":$TARGET_DB_PORT" || true # grep can fail if no connections

        # Trigger packet capture if an issue was detected
        if [ "$TRIGGER_PACKET_CAPTURE" = true ]; then
            log_message "ISSUE DETECTED! Initiating tcpdump for $PACKET_CAPTURE_DURATION
seconds..."
            # Capture on the interface connected to the DB network. Adjust 'eth0' as needed.
            tcpdump -i eth0 -nn -s0 -w "$LOG_DIR/tcpdump_$(date +%Y%m%d%H%M%S).pcap" \
                "host $TARGET_DB_IP and port $TARGET_DB_PORT" &
            TCPDUMP_PID=$!
            sleep "$PACKET_CAPTURE_DURATION"
            kill "$TCPDUMP_PID" || true # Kill tcpdump gracefully
            wait "$TCPDUMP_PID" 2>/dev/null || true # Wait for it to finish
            log_message "Packet capture complete. File: $LOG_DIR/tcpdump_*.pcap"
            TRIGGER_PACKET_CAPTURE=false # Reset flag
        fi

        sleep "$INTERVAL"
    done
    log_message "Connectivity checks finished."
}

# --- Main execution ---
collect_basic_info
run_checks_loop

log_message "Network diagnostics completed. Review logs in $LOG_DIR for intermittent issues."
log_message "Use 'wireshark -r <pcap_file>' to analyze captured packets."
```

This script runs on the web server, continuously checking connectivity to the database. It logs all output and, crucially, triggers a `tcpdump` capture *only when an issue is detected*, saving disk space and making analysis more targeted. This systematic approach is invaluable for tracking down elusive intermittent network problems.

Q49. Describe how you would create a Bash script for automated failover or disaster recovery, specifically focusing on synchronizing data to a secondary site and then switching DNS records. What are the critical considerations for data consistency and minimizing downtime?

Detailed Answer:

Automating failover/DR with Bash requires careful planning to ensure data consistency, minimize RTO (Recovery Time Objective), and prevent split-brain scenarios. This process typically involves data synchronization, validation, and a controlled cutover.

Critical Considerations:

1. Data Consistency (RPO - Recovery Point Objective):

- **Application Quiescence**: Before synchronization, critical applications on the primary site should be gracefully shut down or put into read-only mode to ensure no new writes occur during the sync. This is the ideal but often challenging "zero RPO" scenario.
- **Database Replication**: For databases, rely on built-in replication mechanisms (e.g., PostgreSQL streaming replication, MySQL GTID-based replication). The script would confirm replication lag is zero or acceptable before cutover.
- **File System Sync**: For application data, use tools like `rsync` with `--checksum` and `--delete` (carefully!) for efficient and accurate synchronization. Rsync should be run from primary to secondary.
- **Version Control/Snapshots**: Use versioned storage (e.g., S3 versions, EBS snapshots) for rollback capabilities.

2. Minimizing Downtime (RTO):
   - **Pre-provisioned Secondary Site**: The secondary site should be already deployed and configured, ideally mirroring the primary. This minimizes the time to bring up services.
   - **Automated DNS Updates**: Use cloud provider CLIs (AWS Route 53, Azure DNS, Google Cloud DNS) for programmatic DNS record updates. TTLs should be low (e.g., 60-300 seconds) to ensure quick propagation.
   - **Health Checks**: Implement robust health checks on the secondary site *before* switching DNS to ensure services are fully operational.
   - **Rollback Plan**: Have a clear, automated rollback strategy in case the failover fails or the secondary site exhibits unexpected issues.
3. Preventing Split-Brain:
   - **Primary Site Fencing**: Crucially, ensure the primary site is fully shut down and cannot serve requests *before* the secondary site takes over. This prevents both sites from trying to act as primary, leading to data corruption. This can involve shutting down VMs, unmounting storage, or blocking network access.
   - **Atomic Operations**: Ensure each step of the failover is atomic or reversible.
   - **Locking**: Use a global lock (e.g., a distributed lock service, or a simple flag file on a shared storage) to prevent concurrent failover attempts.

Bash Script Logic:

1. Validate State: Check health of primary and secondary, confirm actual DR scenario (e.g., primary is truly down or unresponsive).
2. Quiesce Primary: Shut down services, stop database writes, unmount shared storage.
3. Final Data Sync: Perform a final `rsync` or confirm database replication is up-to-date.
4. Activate Secondary: Start services on the secondary site, promote database replica to primary.
5. Validate Secondary: Run comprehensive health checks on the secondary site services.
6. Update DNS: Change CNAME or A records to point to the secondary site's load balancer or IP.
7. Post-Failover Checks: Monitor DNS propagation and overall application health on the secondary site.

Production Scenario / Practical Example:

Imagine a simple web application with a PostgreSQL database, where `app.example.com` needs to fail over from `primary-site` to `secondary-site`. DNS is managed by AWS Route 53.

```
#!/bin/bash
# Script: /usr/local/bin/dr_failover.sh

# --- Configuration ---
PRIMARY_SITE_IP="1.2.3.4"
SECONDARY_SITE_IP="5.6.7.8"
DNS_ZONE_ID="Z1ABCDEFGH12345" # Route 53 Hosted Zone ID
DNS_RECORD_NAME="app.example.com"
DNS_RECORD_TYPE="A"
DNS_TTL=60 # Seconds
DB_PRIMARY_HOST="db-primary.example.com" # Or IP
DB_SECONDARY_HOST="db-secondary.example.com" # Or IP
APP_SERVICE_NAME="my-webapp" # systemd service name
LOG_FILE="/var/log/dr_failover.log"
LOCK_FILE="/var/lock/dr_failover.lock"
RSYNC_DATA_PATH="/var/www/html/app_data" # Path to sync
RSYNC_EXCLUDE_FILE="/tmp/rsync_exclude.txt" # Exclude file for rsync

# --- Global Flags ---
FAILOVER_INITIATED=false

# --- Error Handling & Logging ---
set -euo pipefail
exec > >(tee -a "$LOG_FILE") 2>&1 # Redirect all output to log file and stdout

log_message() {
    echo "$(date +'%Y-%m-%d %H:%M:%S') - $1"
}

cleanup() {
    local exit_code=$?
    log_message "Script finished with exit code $exit_code."
    if [ -f "$LOCK_FILE" ]; then
        rm -f "$LOCK_FILE"
        log_message "Lock file $LOCK_FILE released."
    fi
    if [ "$FAILOVER_INITIATED" = true ] && [ "$exit_code" -ne 0 ]; then
        log_message "CRITICAL ERROR: Failover failed. Manual intervention required to prevent
split-brain. Primary site might still be active."
        # Potentially trigger PagerDuty alert here
    fi
}
trap cleanup EXIT INT TERM
```

```
# --- Locking ---
(
    flock -xn 200 || { log_message "ERROR: Another DR failover process is running. Exiting.";
exit 1; }
    log_message "DR Failover lock acquired."

    # --- Step 1: Validate primary site status (Simulated - in real-world, more robust
checks) ---
    log_message "Step 1: Validating primary site status..."
    if ! ping -c 1 -W 1 "$PRIMARY_SITE_IP" >/dev/null; then
        log_message "Primary site $PRIMARY_SITE_IP is unreachable. Proceeding with failover."
    else
        log_message "WARNING: Primary site $PRIMARY_SITE_IP is still reachable. Manual
confirmation needed before proceeding."
        # In a real scenario, this would be a hard stop unless primary is explicitly
confirmed dead.
        # For this example, we'll force it, but this is a critical decision point.
        # exit 1
    fi
    FAILOVER_INITIATED=true

    # --- Step 2: Quiesce primary services (Simulated via SSH) ---
    log_message "Step 2: Quiescing services on primary site..."
    # ssh user@$PRIMARY_SITE_IP "sudo systemctl stop $APP_SERVICE_NAME && sudo systemctl stop
postgresql"
    log_message "Simulated: Services stopped on primary site."

    # --- Step 3: Final data synchronization to secondary (File system & DB) ---
    log_message "Step 3: Performing final data synchronization..."
    # rsync application data
    # echo "node_modules" > "$RSYNC_EXCLUDE_FILE" # Example exclusion
    # rsync -avz --checksum --delete --exclude-from="$RSYNC_EXCLUDE_FILE"
user@$PRIMARY_SITE_IP:$RSYNC_DATA_PATH /mnt/secondary_data/app_data/
    # If rsync fails due to primary unreachability, proceed with existing secondary data.

    # Confirm DB replication lag is zero (Requires database specific CLI/commands, e.g.,
pg_is_in_recovery, pg_current_wal_lsn)
    # ssh user@$DB_SECONDARY_HOST "psql -c 'SELECT pg_is_in_recovery()'" # Should be true
initially
    # ssh user@$DB_SECONDARY_HOST "psql -c 'SELECT pg_last_wal_receive_lsn() =
pg_last_wal_replay_lsn()'" # Should be true for zero lag
    log_message "Simulated: Database replication confirmed to be zero lag."

    # --- Step 4: Activate services on secondary site ---
    log_message "Step 4: Activating services on secondary site..."
    ssh user@$DB_SECONDARY_HOST "sudo pg_ctlcluster 13 main promote" # Promote PostgreSQL
replica
    ssh user@$SECONDARY_SITE_IP "sudo systemctl start postgresql && sudo systemctl start
```

```
$APP_SERVICE_NAME"
    log_message "Services started on secondary site."

    # --- Step 5: Validate secondary site health ---
    log_message "Step 5: Running health checks on secondary site..."
    if ! curl -fsS "http://$SECONDARY_SITE_IP:80/health" >/dev/null; then
        log_message "CRITICAL ERROR: Secondary site health check failed! Aborting failover.
Manual intervention."
        exit 1
    fi
    log_message "Secondary site health checks passed."

    # --- Step 6: Update DNS records ---
    log_message "Step 6: Updating DNS record for $DNS_RECORD_NAME to $SECONDARY_SITE_IP..."
    TEMP_JSON=$(mktemp)
    cat <<EOF > "$TEMP_JSON"
{
  "Comment": "Failover to secondary site",
  "Changes": [
    {
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "$DNS_RECORD_NAME",
        "Type": "$DNS_RECORD_TYPE",
        "TTL": $DNS_TTL,
        "ResourceRecords": [
          {
            "Value": "$SECONDARY_SITE_IP"
          }
        ]
      }
    }
  ]
}
EOF

    aws route53 change-resource-record-sets \
        --hosted-zone-id "$DNS_ZONE_ID" \
        --change-batch file://"$TEMP_JSON"

    rm "$TEMP_JSON"
    log_message "DNS update initiated. Propagation will take up to $DNS_TTL seconds."

    # --- Step 7: Post-failover monitoring (optional, could be external) ---
    log_message "Step 7: Initiating post-failover monitoring. Waiting for DNS propagation..."
    # Loop and check app.example.com points to SECONDARY_SITE_IP
    # This might require external tools or a longer script to confirm.
```

```
    log_message "Failover process completed successfully."

) 200>"$LOCK_FILE" # Associate FD 200 with the lock file
```

This script outlines the major steps, using `flock` for concurrency control, `set -euo pipefail` for error handling, and `trap cleanup` for graceful exit. The critical `FAILOVER_INITIATED` flag helps in identifying partial failovers. Remote commands would require SSH keys for authentication. A robust implementation would replace `ping` with more sophisticated health checks, integrate with specific database promotion tools, and include more comprehensive error recovery logic for each step.

Q50. After a major production incident, you're tasked with collecting data for a Root Cause Analysis (RCA). How would you use Bash scripting to automate the aggregation of relevant logs, system metrics, configuration changes, and potentially code differences from various sources into a structured report?

Detailed Answer:

Automating RCA data collection with Bash significantly reduces the post-incident workload and ensures consistency and completeness. The goal is to gather all pertinent information from disparate sources into a centralized, structured format that facilitates analysis.

Key Data Sources for RCA:

1. Application Logs: From various services, potentially across multiple hosts/containers, often in different formats.

2. System Logs: Kernel logs (`dmesg`), systemd journal (`journalctl`), authentication logs (`auth.log`).

3. Metrics: Snapshots or trends from monitoring systems (e.g., Prometheus queries via `curl` to `/api/v1/query_range`), `vmstat`, `iostat`, `top` output.

4. Configuration Files: `diff` between current and previous versions (e.g., from a Git repo or configuration management system).

5. Deployment History: Records of recent deployments (e.g., Git commit IDs, CI/CD pipeline runs).

6. Process States: `ps auxf`, `lsof`, `ss` output at the time of the incident or immediately after.

7. Code Differences: `git diff` if code changes were deployed recently.

Bash Script Logic:

1. Input Parameters: Incident ID, start/end timestamps of the incident window, affected services/hosts.

2. Directory Structure: Create a well-organized output directory for the RCA data (e.g., `rca_data/<incident_id>/<hostname>/<category>/`).

3. Data Collection Functions: Modular functions for each data type (e.g., `collect_app_logs`, `collect_metrics`, `collect_config_diffs`). These functions would use SSH for remote hosts, `kubectl` for Kubernetes, `curl` for APIs, `git` for repos, etc.

4. Time-Windowed Collection: Most data should be collected within the incident window (e.g., "1 hour before to

30 minutes after").

5. Standardization/Normalization: Where possible, convert logs to a common format or at least filter out noise.

6. Configuration Diffs: Fetch configuration from a source of truth (e.g., Git) and compare with the live configuration.

7. Code Differences: If a recent deployment is suspected, automate `git diff` between the deployed version and the previous one.

8. Summary Generation: After collecting raw data, perform initial aggregations, `grep` for common errors, or generate simple summaries.

9. Archiving: Compress the entire collected data set for long-term storage and easy sharing.

Production Scenario / Practical Example:

An incident occurred on a Kubernetes cluster affecting `api-service` and `worker-service`. The incident window is from `2023-10-26T10:00:00Z` to `2023-10-26T10:30:00Z`.

```
#!/bin/bash
# Script: /usr/local/bin/rca_data_collector.sh

# --- Configuration ---
INCIDENT_ID="${1:-rca_$(date +%Y%m%d%H%M%S)}"
START_TIME_UTC="${2:-$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)}"
END_TIME_UTC="${3:-$(date -u +%Y-%m-%dT%H:%M:%SZ)}"
TARGET_K8S_NAMESPACES="default my-app-namespace" # All namespaces to collect logs from
TARGET_K8S_SERVICES="api-service worker-service" # Specific services/deployments
METRICS_QUERY_URL="http://prometheus-k8s.monitoring.svc.cluster.local:9090/api/v1/query_range
"
OUTPUT_BASE_DIR="/tmp/rca_collection"
RCA_DIR="${OUTPUT_BASE_DIR}/${INCIDENT_ID}"
CONFIG_REPO_PATH="/opt/config-repo" # Local clone of your config Git repo

# --- Error Handling & Logging ---
set -euo pipefail
mkdir -p "$RCA_DIR"
exec > >(tee -a "$RCA_DIR/rca_script.log") 2>&1 # Redirect all output to log file and stdout

log_message() {
    echo "$(date +'%Y-%m-%d %H:%M:%S') - $1"
}

# --- Cleanup function ---
cleanup() {
    local exit_code=$?
    log_message "Script finished with exit code $exit_code."
    if [ "$exit_code" -ne 0 ]; then
        log_message "RCA data collection failed. Please review logs."
    else
```

```
        log_message "RCA data collected successfully in $RCA_DIR"
        log_message "Compressing collected data..."
        tar -czf "${OUTPUT_BASE_DIR}/${INCIDENT_ID}.tar.gz" -C "$OUTPUT_BASE_DIR"
"$INCIDENT_ID"
        rm -rf "$RCA_DIR" # Clean up uncompressed directory
        log_message "Compressed archive: ${OUTPUT_BASE_DIR}/${INCIDENT_ID}.tar.gz"
    fi
}
trap cleanup EXIT INT TERM

# --- Data Collection Functions ---

collect_kubernetes_logs() {
    local namespace="$1"
    local service="$2"
    log_message "Collecting Kubernetes logs for $namespace/$service between $START_TIME_UTC
and $END_TIME_UTC..."
    local output_dir="$RCA_DIR/kubernetes/logs/$namespace/$service"
    mkdir -p "$output_dir"

    # Get all pods for the service
    PODS=$(kubectl get pods -n "$namespace" -l app="$service" -o
jsonpath='{.items[*].metadata.name}' || true)
    if [ -z "$PODS" ]; then
        log_message "No pods found for app=$service in namespace $namespace. Skipping log
collection."
        return
    fi

    for POD in $PODS; do
        log_message "  -> Pod: $POD"
        # kubectl logs --since-time and --until-time are not universally supported across k8s
versions reliably
        # A common approach is to use --since=DURATION and then filter with grep by
timestamp, or fetch all and grep.
        # For simplicity, we'll fetch full logs and grep if needed, or use a duration that
spans the incident.
        kubectl logs "$POD" -n "$namespace" --since "2h" > "$output_dir/${POD}_full.log" ||
true
        # If logs are JSON, you might want to `jq .` here.
        log_message "  -> Logs for $POD saved."
    done
}

collect_kubernetes_configs() {
    local namespace="$1"
    local service="$2"
    log_message "Collecting Kubernetes configurations for $namespace/$service..."
```

```
    local output_dir="$RCA_DIR/kubernetes/configs/$namespace/$service"
    mkdir -p "$output_dir"

    # Collect deployment, service, configmap, secrets (metadata only)
    kubectl get deployment "$service" -n "$namespace" -o yaml > "$output_dir/deployment.yaml"
|| true
    kubectl get service "$service" -n "$namespace" -o yaml > "$output_dir/service.yaml" ||
true
    # For ConfigMaps/Secrets, consider only collecting specific ones or metadata.
    # kubectl get cm -n "$namespace" -l app="$service" -o yaml >
"$output_dir/configmaps.yaml" || true
    log_message "  -> Kubernetes configs for $namespace/$service saved."
}

collect_metrics() {
    log_message "Collecting Prometheus metrics snapshots between $START_TIME_UTC and
$END_TIME_UTC..."
    local output_dir="$RCA_DIR/metrics"
    mkdir -p "$output_dir"

    # Example: CPU utilization for a service
    # Prometheus queries expect Unix timestamps for start/end
    local START_TIMESTAMP=$(date -d "$START_TIME_UTC" +%s)
    local END_TIMESTAMP=$(date -d "$END_TIME_UTC" +%s)
    local STEP="30s" # 30-second resolution

    for service in $TARGET_K8S_SERVICES; do
        log_message "  -> Querying CPU usage for $service..."
        # Example query: sum of container_cpu_usage_seconds_total by instance and namespace
        # Adjust query to your specific metrics and labels
        local QUERY="sum(rate(container_cpu_usage_seconds_total{namespace='my-app-namespace',
pod=~'${service}.*'}[5m])) by (pod)"
        local ENCODED_QUERY=$(echo "$QUERY" | jq -sRr @uri) # URL encode the query

        # Use curl to query Prometheus API
        # -g disables URL globbing for curl, important for brackets in query
        curl -g
"${METRICS_QUERY_URL}?query=${ENCODED_QUERY}&start=${START_TIMESTAMP}&end=${END_TIMESTAMP}&st
ep=${STEP}" \
            > "$output_dir/${service}_cpu_usage.json" || true
        log_message "  -> CPU usage for $service saved."
    done
}

collect_config_repo_diffs() {
    log_message "Collecting configuration repository differences..."
    local output_dir="$RCA_DIR/config_diffs"
    mkdir -p "$output_dir"
```

```
    if [ ! -d "$CONFIG_REPO_PATH" ]; then
        log_message "WARNING: Config repository path '$CONFIG_REPO_PATH' not found. Skipping
config diffs."
        return
    fi

    # Assuming a git repo. Fetch latest.
    (
        cd "$CONFIG_REPO_PATH"
        git fetch origin || true
        git pull origin main || true # Update local repo

        # Get recent commits within the incident window
        log_message "  -> Recent commits in config repo:"
        git log --since="$START_TIME_UTC" --until="$END_TIME_UTC" --pretty=format:"%h - %an,
%ar : %s" > "$output_dir/recent_commits.txt" || true

        # Find the last commit before the incident start time (or a known baseline)
        LAST_GOOD_COMMIT=$(git log --until="$START_TIME_UTC" --pretty=format:"%H" -n 1 ||
true)
        CURRENT_HEAD=$(git rev-parse HEAD || true)

        if [ -n "$LAST_GOOD_COMMIT" ] && [ "$LAST_GOOD_COMMIT" != "$CURRENT_HEAD" ]; then
            log_message "  -> Generating diff between $LAST_GOOD_COMMIT (before incident) and
$CURRENT_HEAD (current state)."
            git diff "$LAST_GOOD_COMMIT" "$CURRENT_HEAD" >
"$output_dir/config_diff_pre_incident_to_current.patch" || true
        else
            log_message "  -> No significant config changes detected around incident or
baseline not found."
        fi
    )
}

# --- Main Execution ---
log_message "Starting RCA data collection for incident: $INCIDENT_ID"
log_message "Incident Window: $START_TIME_UTC to $END_TIME_UTC"

# 1. Collect Kubernetes logs and configs
for ns in $TARGET_K8S_NAMESPACES; do
    for svc in $TARGET_K8S_SERVICES; do
        collect_kubernetes_logs "$ns" "$svc" &
        collect_kubernetes_configs "$ns" "$svc" &
    done
done
wait # Wait for all background log/config collection to finish
```

```
# 2. Collect Prometheus metrics
collect_metrics &
wait

# 3. Collect configuration repository differences
collect_config_repo_diffs &
wait

log_message "All data collection tasks initiated. Script will now clean up and compress."
```

This script leverages `kubectl` for Kubernetes-specific data, `curl` for Prometheus, and `git` for configuration analysis. It runs collection tasks in parallel using `&` for speed and uses `wait` to ensure all processes complete before archiving. The `trap cleanup` ensures the data is always compressed and the temporary directory removed, even if the script fails. This structured approach provides a comprehensive dataset for a thorough RCA.